

License: arXiv.org perpetual non-exclusive license
arXiv:2602.23784v1 [cs.LG] 27 Feb 2026

TradeFM: A Generative Foundation Model for Trade-flow and Market Microstructure

Maxime Kawawa-Beaudan

Srijan Sood

Kassiani Papasotiriou

Daniel Borrajo

Manuela Veloso

Abstract

Foundation models have transformed domains from language to genomics by learning general-purpose representations from large-scale, heterogeneous data. We introduce TradeFM, a 524M-parameter generative Transformer that brings this paradigm to market microstructure, learning directly from billions of trade events across $> 9K$ equities. To enable cross-asset generalization, we develop scale-invariant features and a universal tokenization scheme that map the heterogeneous, multi-modal event stream of order flow into a unified discrete sequence – eliminating asset-specific calibration. Integrated with a deterministic market simulator, TradeFM-generated rollouts reproduce key stylized facts of financial returns, including heavy tails, volatility clustering, and absence of return autocorrelation. Quantitatively, TradeFM achieves $2\text{--}3\times$ lower distributional error than Compound Hawkes baselines and generalizes zero-shot to geographically out-of-distribution APAC markets with moderate perplexity degradation. Together, these results suggest that scale-invariant trade representations capture transferable structure in market microstructure, opening a path toward synthetic data generation, stress testing, and learning-based trading agents.

Foundation Model, Generative Model, Market Microstructure, Algorithmic Trading, Trade-flow, Financial Time Series, Reinforcement Learning, Multi-Agent Simulation, Transformer, TradeFM

J.P. Morgan AI Research, New York, NY, USA

1 Introduction

Financial markets are complex adaptive systems where the strategic interactions of heterogeneous participants give rise to high-frequency, non-stationary dynamics (Bouchaud, [2010](#)). The atomic record of these interactions is **order flow**: the stream of buy and sell orders submitted to the market (Sirignano & Cont, [2021](#)). Modeling order flow is a formidable challenge – the statistical properties of trade data vary dramatically across assets, liquidity regimes, and time periods (Pasca, [2015](#)), and any single participant observes only a partial view of the true market state.

Despite this complexity, there is strong evidence that price formation follows *universal* principles across markets. Sirignano & Cont ([2021](#)) showed that a single deep learning model trained on pooled multi-stock data outperforms asset-specific models, even for held-out stocks. This finding motivates a natural question: *can a single foundation model learn a general-purpose representation of market mechanics from raw, multi-asset order flow?*

We answer in the affirmative with **TradeFM**, a 524M-parameter decoder-only Transformer pre-trained on over 10 billion tokens from > 9K US equities. Our contributions are fourfold:

1. **TradeFM**: A large-scale generative foundation model for market microstructure that learns unified trade-flow dynamics from billions of transactions across the breadth of the US equity market.
2. **Learning from Partial Observations**: Unlike prior deep learning approaches that require the full limit order book as input (Table [1](#)), TradeFM learns from a partially observed market state – the event stream available to any single participant – demonstrating that a foundation model for microstructure does not require privileged access to the full order book.
3. **Scale-Invariant Representation and Tokenization**: An end-to-end methodology that transforms raw, heterogeneous trade data into a unified discrete sequence via scale-invariant features and a universal tokenization scheme, enabling a single model to generalize across diverse assets and liquidity regimes without asset-specific calibration.

4. **Closed-Loop Market Simulation:** Integration of the pre-trained model with a deterministic market simulator, creating a high-fidelity environment for evaluating realism via stylized fact reproduction, studying market impact, and training learning-based agents.

In closed-loop evaluation, TradeFM-generated rollouts reproduce canonical stylized facts of financial returns. Quantitatively, it achieves $2\text{--}3\times$ lower distributional error than Compound Hawkes baselines and generalizes zero-shot to out-of-distribution APAC markets with moderate perplexity degradation.

2 Background

2.1 The Mechanics of Modern Electronic Markets

To provide context for a general AI/ML audience, we briefly introduce the core concepts of market microstructure fundamental to this work, which are standard features of modern electronic markets (Hasbrouck, [2007](#)).

Financial markets are predominantly organized around a **Limit Order Book (LOB)**, a real-time record of all outstanding orders for a security that functions as a continuous, double-sided auction. It consists of a **bid** (buy) side and an **ask** (sell) side; the midpoint between the highest bid and the lowest ask is an asset's **mid-price**. The ease with which an asset can be bought or sold quickly at a stable price is the asset's **liquidity**.

Market participants interact with the LOB through a sequence of actions, collectively known as **order flow**. Participants may submit **limit orders** with a specific price limit, which sit on the book waiting to be matched. The distance between the **order price** and the midprice is the **price depth**, quoted in **ticks** (the minimum price increment, typically \$0.01) or **basis points** (bps; 0.01% of the price). They may also submit **market orders** for immediate execution against resting limit orders starting at the best bid/ask, and **cancellations** to withdraw resting orders. When an incoming order is matched with a resting one, a **fill** (or trade execution) occurs. This matching process is generally governed by a deterministic **price-time priority** algorithm, where orders are first prioritized by price and then by time of submission. These elements and mechanisms constitute **market microstructure**.

2.2 Stylized Facts as Emergent Properties

The strategic interactions of market participants give rise to endogenous market dynamics (Bouchaud, [2010](#)). These dynamics, in turn, give rise to universal and persistent statistical properties known as **stylized facts**. These facts are observed across a wide range of assets, markets, and time periods, and serve as a crucial benchmark for the realism of any generative market model (Cont, [2001](#); Ratliff-Crain et al., [2025](#)). Key stylized facts include:

- **Heavy-Tailed Returns:** Returns are leptokurtic – extreme movements occur far more frequently than a Gaussian predicts.
- **Volatility Clustering:** High-volatility periods cluster together, manifesting as slowly decaying autocorrelation of absolute returns.
- **Lack of Autocorrelation in Returns:** Consistent with efficient markets, return autocorrelation is insignificant beyond very short lags.

3 Related Work

The modeling of market microstructure has evolved from explicit, theory-driven formulations toward implicit, data-driven representation learning. Our work continues this trajectory, positioning a generative foundation model as the natural next step to learn universal market dynamics directly from raw, heterogeneous data.

3.1 Market Microstructure and Order Flow Modeling

Classical Stochastic Models

A significant body of literature models order arrival times using point processes, such as Hawkes processes, to capture the self-exciting nature of order flow (Bacry et al., [2015](#)). More sophisticated approaches adopt Compound Hawkes processes, which combine Hawkes processes to model interarrival times with other fitted empirical distributions to model additional features like volumes and price depths (Jain et al., [2024](#)). While providing strong theoretical grounding, these models rely on specific parametric assumptions (e.g., Gaussianity) that are unable to capture the heavy-tailed nature of market returns. Similarly, ensemble methods based on Hidden Markov Models have been applied to classify sequences of trade activity, demonstrating that probabilistic sequence models capture meaningful structure in financial data even with limited labeled examples (Kawawa-Beaudan et al., [2024](#)).

Agent-Based Models

Agent-based models (ABMs) simulate market dynamics by defining the behavior of individual participants and observing the emergent properties of the system (Byrd et al., [2020](#)). While

ABMs have historically required hand-crafting agent behaviors, recent approaches have shown success in calibrating agents on real market data (Dwarakanath et al., 2024). Our work contributes to this line of research by enabling the learning of complex market dynamics, which can serve as a foundation for more sophisticated agent-based modeling.

Early Deep Learning Models

The application of deep learning to LOB data was pioneered by models like DeepLOB (Zhang et al., 2019). These models demonstrated the potential of learning features directly from data but were typically trained on a subset of instruments. This limits their ability to learn universal representations across diverse assets and market conditions.

3.2 Transformers and Foundation Models in Finance

The Transformer architecture (Vaswani et al., 2017) has been widely applied across domains including genomics (Ji et al., 2021), time-series forecasting (Wen et al., 2023), and payment transaction modeling (Raman et al., 2024). In market microstructure, recent transformer models focus on discriminative prediction for short-term forecasting (Berti & Kasneci, 2025; Xiao et al., 2025). These approaches operate on full limit order book snapshots and target single or few instruments. Generative adversarial networks (Wiese et al., 2020) and diffusion-based approaches have also been applied to financial data generation. These methods typically target price or return series rather than event-level order flow, and have not been evaluated at multi-asset scale.

A prominent generative approach is MaRS (Li et al., 2025), a market simulator with a foundation model backbone. While building on design principles from Li et al. (2025), TradeFM distinguishes itself in three dimensions (Table 1): (1) pre-training data explicitly constructed to **maximize heterogeneity** across thousands of assets, sectors, and liquidity regimes; (2) **partial observability** – learning from Level 3 trade messages rather than full LOB snapshots, matching the information available to real market participants; and (3) **zero-shot geographic generalization** to unseen markets, enabled by scale-invariant feature design rooted in Universal Price Formation theory (Sirignano & Cont, 2021). Direct quantitative comparison with MaRS is infeasible: it requires limit order book context as input (strictly more information than event streams), is trained on a different market (Chinese equities), and covers fewer assets (~500 vs. > 9K).

Model	Input	Assets	Params	Zero-Shot
-------	-------	--------	--------	-----------

DeepLOB	Full LOB	5	60K	No
LOBS5	Full LOB	2	6.3M	No
MaRS	Full LOB	500	~1B	No
TradeFM	Event stream	> 9K	524M	Yes

Table 1: **Comparison with related microstructure models.** TradeFM uniquely combines partial observability, large-scale multi-asset training, and zero-shot geographic generalization.

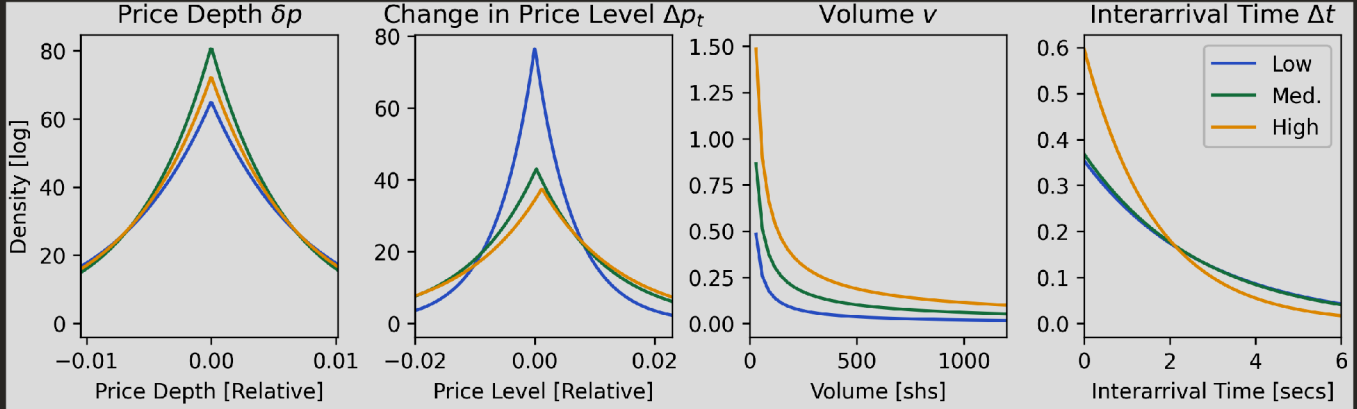


Figure 1: **Trade feature distributions.** Canonical distributions for core trade features, conditioned on liquidity. Price features are leptokurtic (Laplace); volume follows a heavy-tailed power-law; and interarrival time is exponential.

4 Problem Formulation

We formulate the task of modeling market microstructure as a generative, autoregressive sequence modeling problem. Let the market dynamics be represented by a sequence of discrete trade events, $E = (e_1, e_2, \dots, e_T)$. The objective is to learn the conditional probability distribution $P(e_t | e_{<t})$, where $e_{<t}$ denotes the sequence of events preceding e_t . By learning this distribution, the model can generate realistic sequences of future trade events, simulating the market’s evolution.

4.1 Trade Event Representation

A single trade event e_t is a multi-feature tuple capturing the state of the market at the moment of a transaction. Formally, an event is represented as $e_t = (\Delta t_t, \delta p_t, v_t, a_t, s_t)$, where the core features are: Δt_t : Interarrival time since the previous event (seconds); δp_t : Price depth of the

transaction (basis points); v_t : Volume of the transaction (shares); a_t : The action/order type (e.g., order submission, cancellation); s_t : The side of the initiating order (buy/bid or sell/ask). The distributions of these features are depicted in Figure 1.

4.2 Key Technical Challenges

Modeling this data stream presents challenges inherent to high-frequency markets: the **Heterogeneity and Distribution Shift** across diverse assets and varying time periods; the **Sparsity and Irregularity** of the asynchronous event stream; the **Partial Observability** of the true market state from transaction data; and a **High-Dimensional, Multi-Modal Feature Space** combining continuous and categorical values.

5 Data and Feature Engineering

Our methodology is designed to process raw, heterogeneous transaction data at scale and transform it into a standardized format suitable for a generative foundation model. This pipeline consists of data curation, robust feature engineering, and a novel tokenization scheme.

5.1 Data Sources and Scale

The model is pre-trained on a proprietary dataset built from billions of raw, tick-level US equities transactions, spanning 368 trading days from February 2024 to September 2025, across the breadth of the US equities market. This represents over 19 billion tokens across 1.9 million date-asset pairs. We employ a temporal hold-out strategy, reserving January 2025 onward across all assets for the test set, yielding a training set of 10.7 billion tokens and a test set of 8.7 billion tokens. The tokenizer is calibrated on the first 30 days of the training data, February 2024. For evaluating out-of-distribution generalization we also hold out one month of data from APAC regions, namely Jan. 2025, for both Japan and China.

5.2 Mid-Price Estimation

A robust estimate of the true market mid-price (p_t^{mid}) is critical for normalizing price-related features. In our partial-information setting, the mid-price is unobserved; we only see noisy execution prices (p_t^{exec}). Building on the classical Volume-Weighted Average Price (Berkowitz et al., 1988), we introduce **Exponentially-Weighted VWAP (EW-VWAP)**, a time-aware estimator that jointly accounts for trade volume and recency: $\hat{p}_t^{\text{EW-VWAP}} = \text{EMA}(p_t^{\text{exec}} \cdot v_t) / \text{EMA}(v_t)$. The smoothing factor α is determined by a time-based halflife, ensuring that the estimate gives more weight to larger and more recent trades, providing a stable and representative

price benchmark. Unlike naive rolling averages, EW-VWAP is comparable across assets with vastly different liquidity profiles (see Appendix [A.3](#), Figure [8](#)).

5.3 Scale-Invariant Feature Construction

The statistical properties of trading data vary widely across sectors, liquidity profiles, and nominal prices. In raw dollar, share, and second terms, price depths, volumes, and interarrival times for an asset like AAPL may differ greatly from those of a penny stock. Trade representations must therefore be carefully designed to enforce homogeneity across assets. Sirignano & Cont ([2021](#)) demonstrate that price formation follows universal principles across stocks: universal models trained on all assets outperform asset-specific models, even for held-out stocks, suggesting deep learning can learn appropriate normalizations from raw data. Building on this, we explicitly construct scale-invariant features to enforce homogeneity, extending universality from order book contexts to heterogeneous event streams across diverse trading venues and market structures.

- **Interarrival Time (Δt_t):** Wall clock time since the previous event: $w_t - w_{t-1}$, in seconds.
- **Log-Transformed Volume (v_t):** To compress the wide dynamic range of order sizes, which follow heavy-tailed, power-law distributions (Vyetrenko et al., [2020](#)), we apply a logarithmic transformation: $v_t = \log(1 + V_t)$ where V_t is the raw share volume.
- **Normalized Price Depth (d_t):** The depth of a limit order with order price p_t^{order} , relative to the mid-price: $d_t = \frac{p_t^{\text{order}} - \hat{p}_t^{\text{mid}}}{\hat{p}_t^{\text{mid}}}$, normalizing the raw δp_t from the event tuple. This representation is comparable across differently priced assets, unlike prior work using price depths in ticks.
- **Relative Price Level vs. Open (Δp_t):** To capture intraday market movement, we quote the current mid-price relative to the day's opening price (p_0): $\Delta p_t = \frac{\hat{p}_t^{\text{mid}} - p_0}{p_0}$.

While price-related features are computed as unit-less ratios, we refer to them in terms of basis points (bps) for interpretability, where a ratio of 0.01 corresponds to 100 bps. Figure [7](#) (Appendix [A](#)) demonstrates distributional stationarity of relative features (vs. tick based). Figure [12](#) (Appendix [D.3](#)) demonstrates the subsequent temporal stability of scale-invariant features.

5.4 Market and Participant-Level Sequences

In downstream applications, TradeFM can be used to model the behavior of the entire market (e.g., for synthetic data generation) or that of individual participants (e.g., for agent-based modeling). To support this, our training data includes sequences aggregated at both the market level and the participant level, with approximately a 1.6:1 market-to-participant ratio at the token level. The model receives a binary indicator feature, I_{MP} , to distinguish between these two contexts.

6 Tokenization

Standard Transformer architectures, as applied in natural language processing, operate on univariate sequences where each element is a single token from a discrete vocabulary. Our trade event data is a sequence of multi-feature tuples, each comprising a mix of continuous and categorical values. The core challenge of tokenization is to map this event stream into a univariate discrete sequence.

6.1 Binning Strategy and Outlier Handling

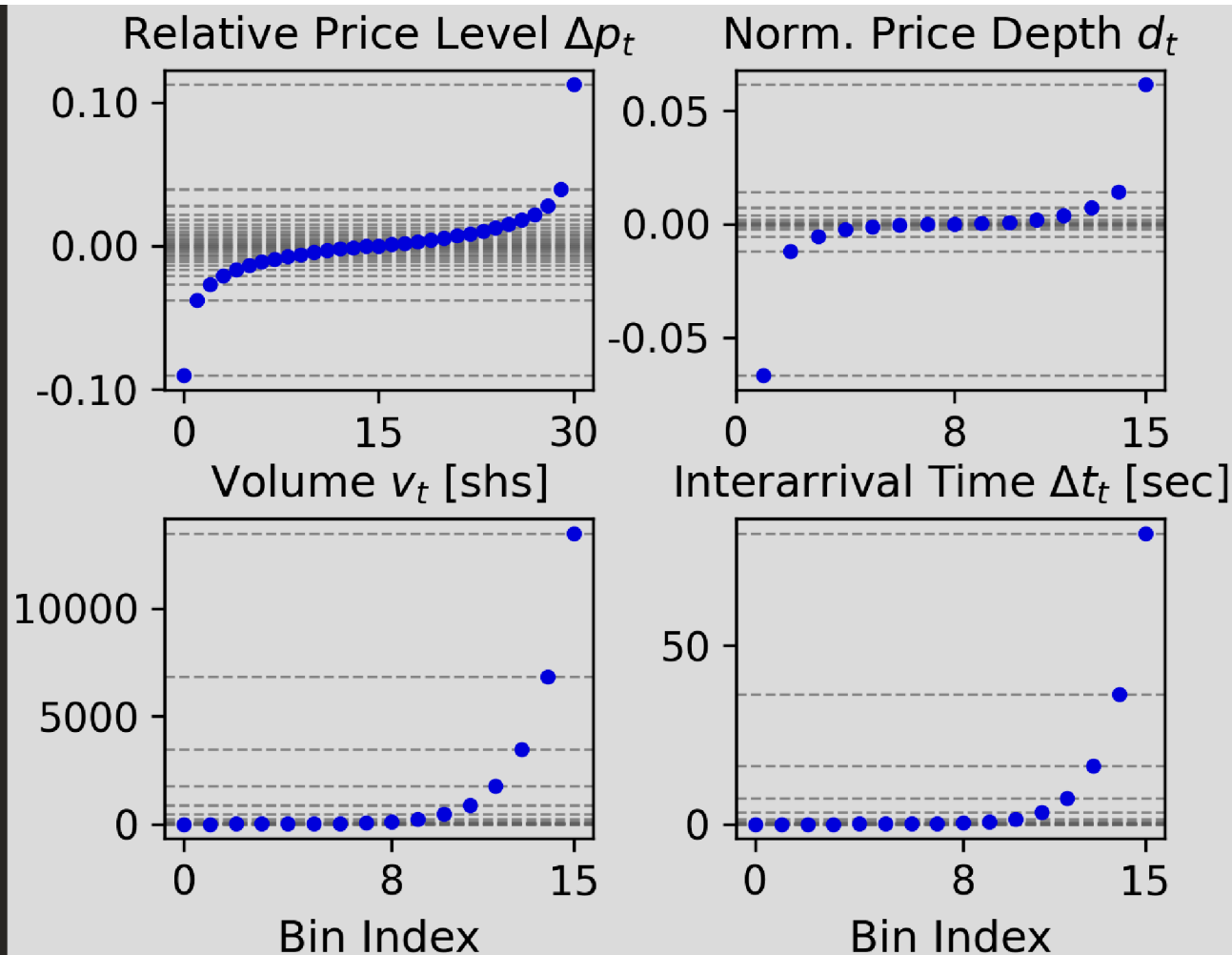


Figure 2: **Calibrated bin edges.** Price features (top) use quantile-based binning for high resolution near the mean; volume and time (bottom) use logarithmic bins to capture their wide dynamic range.

We discretize each continuous feature by partitioning its distribution into a fixed number of bins. For price-related features, which have symmetric but highly peaked distributions, we employ **Equal-Frequency Binning** (quantile-based). This ensures that the bins in the dense central region of the distribution have a higher resolution, while still capturing the less frequent values in the tails. For log-transformed features, like volume and interarrival time, we use **Equal-Width Binning**. Applying equal-width bins to the logarithmic values creates bins that are effectively logarithmic in the original feature space, providing a way to represent values that span multiple orders of magnitude.

This hybrid approach ensures a relatively uniform token distribution, preventing the model from wasting capacity on rare tokens or nearly-empty bins. Before binning, we exclude outliers above the 99th percentile for each feature, and additionally exclude outliers below the 1st

percentile for price depth and price level. We reserve special bins to represent these out-of-range values to prevent the model from allocating excessive capacity to extremely rare events. We calibrate this tokenizer on the first 30 days of our data. The calibrated bin edges are shown in Figure 2.

6.2 Multi-Feature Token Composition

While our model’s input at each time step is multi-featured, the decoder is trained to predict a single, unidimensional token, representing the core trade event. Thus, we combine the discrete bin indices of the trade-related features, $(i_{\Delta t}, i_{\delta p}, i_v, i_a, i_s)$, into a single composite integer, i_{trade} .

This is accomplished by treating the feature indices as digits in a mixed base number system. Each feature’s bin index is a “digit”, and the number of possible values for the subsequent features acts as the “base” at each position. For a concrete example of the encoding process, see Section A.4. The number of bins for each feature is: $n_{\delta p} = 16$ for price depth, $n_v = 16$ for volume, $n_{\Delta t} = 16$ for interarrival time, $n_s = 2$ (buy, sell) for side, and $n_a = 2$ (add or cancel order) for action type. The composite trade token i_{trade} , a single integer encoding all constituent features, is calculated as:

$$\begin{aligned} i_{\text{trade}} = & (i_a \times n_s \times n_{\delta p} \times n_v \times n_{\Delta t}) + \\ & (i_s \times n_{\delta p} \times n_v \times n_{\Delta t}) + \\ & (i_{\delta p} \times n_v \times n_{\Delta t}) + (i_v \times n_{\Delta t}) + i_{\Delta t} \end{aligned} \quad (1)$$

This yields a vocabulary size of 16,384 for the predictable trade tokens. The model input at each time step is a tuple containing this token along with several non-predicted features used for conditioning. These contextual features are provided as separate inputs, and are not part of the trade token i_{trade} calculated in Equation 1. The contextual features are:

- i_l : The liquidity bin index ($n_l = 3$), determined by binning each asset into low, medium, or high liquidity ranges based on its Average Daily Volume (ADV).
- $i_{\Delta p_t}$: The price level change bin index ($n_{\Delta p_t} = 32$).
- I_{MP} : A binary indicator for market-level vs. participant-level sequences.

The final input is $[i_l, I_{MP}, i_{\Delta p_t}, i_{\text{trade}}]$. This formulation allows the model to be conditioned on broader market context while focusing its predictive power on the next trade event.

7 TradeFM Architecture

TradeFM is a decoder-only Transformer, trained from scratch with a custom configuration. The architecture is based on the Llama family and incorporates enhancements including grouped-query-attention (GQA) and rotary positional encoding (RoPE) (Touvron et al., [2023](#)). Our model size is **524 million parameters**, a size chosen based on Chinchilla scaling laws (Hoffmann et al., [2022](#)) for our dataset size (see Appendix [B.2](#) for detailed hyperparameter choices). The model is trained on **3 Nvidia A100 GPUs**; we include detailed training setup details in Appendix [B.3](#).

7.1 Tabular Input Embedding

We employ a tabular embedding approach to handle our multi-feature input tokens (as described in Section [6.2](#)). Each feature in the input tuple $[i_t, I_{MP}, i_{\Delta p_t}, i_{\text{trade}}]$ is first projected into its own embedding space using an embedding table. These embedding vectors are concatenated and passed through a linear projection layer to create a unified representation in the model's hidden dimension.

8 Market Simulator

To evaluate the realism of our generative model, we require an environment that can simulate the market execution of a sequence of predicted trades. We build a lightweight, deterministic simulator tailored to our specific experimental setting. Our simulator serves two critical functions: 1) it provides the dynamic, state-dependent price level features required by our model during generative rollouts, and 2) it allows us to test if the model's generated trade flow can reproduce the well-known stylized facts of asset returns.

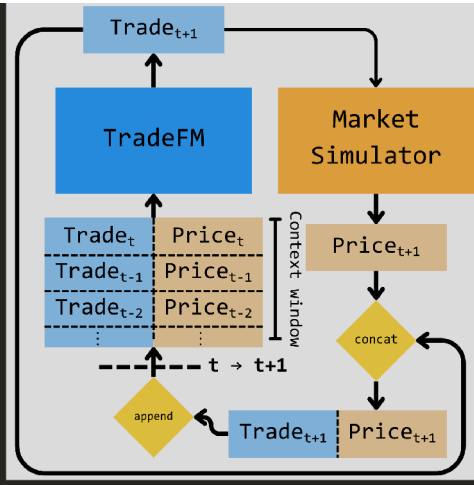


Figure 3: **Closed-loop simulation architecture.** TradeFM predicts a trade, the Market Simulator executes it, and the updated market state is fed back to the model.

8.1 Deterministic Design

The simulator is designed to mimic core mechanics of a modern electronic exchange. It maintains a limit order book (LOB) for an asset, an internal clock, and an estimate of the market mid-price (the midpoint of the best bid and ask). The order matching engine employs **price-time priority**: incoming orders are matched with the best-priced order on the opposite side of the book, with ties in price broken by selecting the earliest-submitted resting order (Nasdaq Listing Center, 2024). Before using the simulator to validate our large trade model, **we validate the realism of the simulator itself via stylized facts** discussed in the Appendix D.2.

8.2 The Closed-Loop Rollout

The simulator creates a closed-loop system where the model and environment interact dynamically. This process, which we term a *rollout*, is shown in Figure 3 and proceeds as follows:

1. **Prediction:** Given a history of market events, TradeFM predicts the next event token, i_{trade} . We use multinomial sampling with a repetition penalty of 1.2 to decode the token.
2. **Execution:** The predicted event (e.g., order or cancellation) is passed to the simulator, which executes it against the LOB according to its price-time priority rules.
3. **State Update:** The simulator updates its internal state, including the LOB and the mid-price.

4. **Feedback:** The market state is used to generate the contextual features for the next time step, which are appended to the history and fed back into TradeFM to generate the next prediction.

This recursive loop allows us to generate long, dynamic sequences of market activity. Crucially, it enables the study of second-order effects like price impact, as the model’s own predictions influence the market state that conditions its future predictions.

9 Experiments

Evaluating generative models of financial markets presents a unique challenge due to the non-stationary nature of the data. Rather than relying on next-token perplexity, we evaluate TradeFM by its ability to reproduce the invariant stylized facts of market behavior (Section 2.2). A model that generates synthetic data exhibiting these facts has learned the time-invariant structure of the market, not merely memorized historical patterns. We conduct three experiments: (1) **stylized fact reproduction**, (2) **distributional fidelity** of generated order flow, and (3) out-of-distribution **generalization and controllability**.

Baselines

We compare against a **calibrated Zero-Intelligence (ZI) agent** (Gode & Sunder, 1993; Farmer et al., 2005) and a **Compound Hawkes** process (Bacry et al., 2015; Jain et al., 2024) that models trade arrivals via self-exciting dynamics with fitted volume and price distributions. Both baselines interact with the same simulator and evaluation pipeline. Details in Appendix B.6. We adopt ZI and Compound Hawkes as baselines because they operate on trade-level event streams, matching our partial observability setting. Neural generative models evaluated in LOB-Bench (Nagy et al., 2025) require full limit order book input, precluding direct comparison.

9.1 Experiment 1: Stylized Fact Reproduction

To validate the realism of the generated market trajectories, we evaluate their ability to reproduce key stylized facts of log returns ($r_{t,\Delta t}$). We generate 10 rollouts of 1,024 events for 9 assets across 3 liquidity tiers, for each of 9 held-out months, conditioned on a context of 1,024 real historical events. For reference, 1,024 events correspond to approximately 2–5 minutes of market time for high-liquidity assets (e.g., AAPL), 15–60 minutes for medium-liquidity, and 1–4 hours for low-liquidity assets. We compute autocorrelations over time lags τ , and evaluate kurtosis over return intervals Δt_r .

Δt_r	Kolmogorov-Smirnov			Wasserstein		
	ZI	Hawkes	TradeFM	ZI	Hawkes	TradeFM
10	0.376	0.039	0.013	0.0006	0.0004	0.0001
30	0.439	0.075	0.024	0.0015	0.0012	0.0002
60	0.429	0.101	0.035	0.0027	0.0023	0.0004
120	0.429	0.098	0.043	0.0043	0.0043	0.0007

Table 2: **Log return distributional fidelity.** Mean K-S and Wasserstein distances from real data across return intervals Δt_r (sec), averaged over 9 assets, 3 liquidity tiers, and 9 held-out months. TradeFM achieves 2–3 $\times$ lower K-S distance than Compound Hawkes.

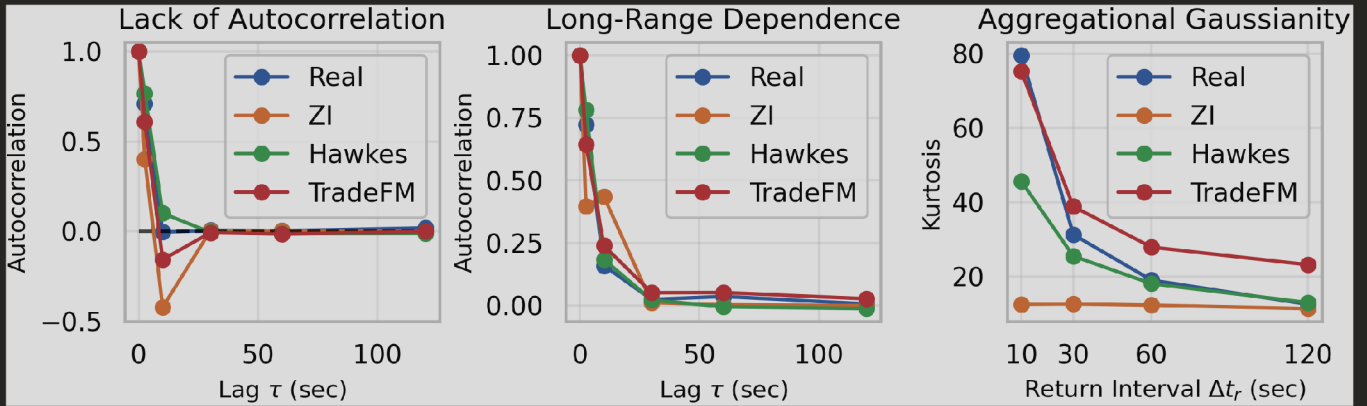


Figure 4: **TradeFM model validation.** Simulated returns exhibit: (Left) near-zero autocorrelation, (Middle) slowly decaying autocorrelation of absolute returns (volatility clustering), and (Right) heavy tails and aggregational Gaussianity.

Results

As summarized in Figure 4 and Table 2, our simulations successfully reproduce the target stylized facts, demonstrating a close correspondence with real market data:

- **Lack of Autocorrelation:** The autocorrelation of simulated log returns (left panel) quickly decays to statistically insignificant levels as the lag τ increases. This is consistent with the efficient market hypothesis. The ZI baseline exhibits spurious autocorrelation.
- **Long-Range Dependence:** The autocorrelation of absolute log returns (middle panel) decays slowly, indicating that our model has captured the long-memory nature of volatility clustering.

- **Heavy Tails:** The kurtosis of simulated returns (right) is high for short time scales (Δt_r), confirming the presence of heavy tails. TradeFM most faithfully captures the leptokurtic nature of returns and its decay across time scales.
- **Aggregational Gaussianity:** As Δt_r increases, the kurtosis of TradeFM correctly approaches that of a normal distribution, capturing the reversion towards normality over longer time horizons.

Table 2 quantifies these results via the Wasserstein distance (W_1) and Kolmogorov-Smirnov (K-S) statistic between real and generated log return distributions. TradeFM outperforms all baselines in both metrics across return intervals.

9.2 Experiment 2: Quantitative Fidelity

While stylized facts confirm that the model captures emergent market dynamics, they do not assess how well the generated order flow aligns with reality. We conduct a quantitative evaluation of distributional fidelity, adopting frameworks established in recent benchmarks for generative order flow models (Nagy et al., 2025). As in the benchmark, we mean-variance normalize distributions before computing Wasserstein distance to make this metric comparable between quantities.

Using the rollouts described in Section 9.1, we compute the Kolmogorov-Smirnov statistic and Wasserstein distance between real and generated distributions for key microstructure variables, including Order Volume, Interarrival Times, Bid-Ask Spreads, and Order Book Imbalance. This evaluation averages results across 9 assets, 3 liquidity tiers, and 9 held-out months. As shown in Table 3, TradeFM achieves lower distance metrics than baseline approaches on most quantities, demonstrating superior fidelity in reproducing the statistical properties of market data. The notable exception is bid-ask spreads, where the Compound Hawkes baseline achieves lower K-S distance. We attribute this to the Hawkes process explicitly modeling inter-arrival dynamics that directly govern spread formation, whereas TradeFM’s spread emerges indirectly from the simulator’s deterministic order matching logic. This suggests that hybrid approaches combining learned order flow with explicit spread modeling may yield further improvements. We provide detailed results in Appendix D.3, Figure 14.

The closed-loop evaluation couples model quality with simulator fidelity. Two lines of evidence partially disentangle their contributions: the ZI control through the identical simulator fails to reproduce stylized facts, and Table 3 evaluates order flow directly before simulator interaction. Full disentanglement remains future work.

Quantity	Kolmogorov-Smirnov			Wasserstein		
	ZI	Hawkes	TradeFM	ZI	Hawkes	TradeFM
Spreads	0.400	0.218	0.238	0.375	0.302	0.400
Interarrival Times	0.651	0.515	0.281	0.415	0.626	0.318
Price Depth	0.436	0.281	0.169	0.390	0.348	0.339
Order Book Imbalance	0.237	0.155	0.142	0.200	0.165	0.099
Bid Volume	0.460	0.296	0.386	0.616	0.278	0.130
Ask Volume	0.391	0.380	0.360	0.638	0.198	0.160

Table 3: **Distributional fidelity of generated order flow.** Mean K-S and Wasserstein distances across 9 assets, 3 liquidity tiers, and 9 held-out months. TradeFM achieves lowest distance on most quantities. Per-month breakdowns in Appendix [D.3](#), Figure [14](#).

9.3 Experiment 3: Generalization and Controllability

To validate our claim that TradeFM learns a universal grammar of market microstructure that generalizes beyond the assets and time periods seen during training, we perform extensive out-of-distribution (OOD) evaluations.

Temporal and Geographic Robustness

Financial markets are non-stationary, with regimes shifting over time. We evaluate model performance over a hold-out period from January–September 2025, a period exhibiting heightened volatility distinct from the 2024 training set. As detailed in Appendix [D.3](#), Figure [14](#), distance metrics remain stable over this 9-month horizon.

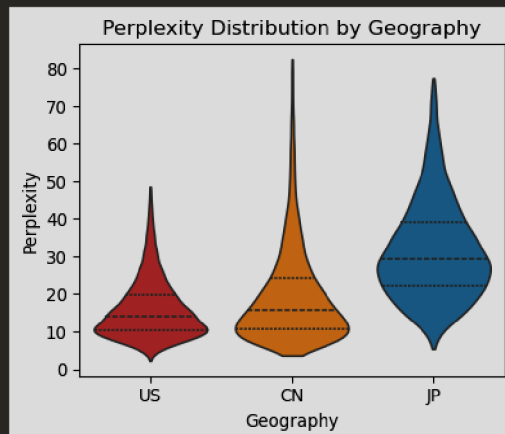


Figure 5: **Zero-shot geographic generalization.** Perplexity distributions for TradeFM (trained on US equities) evaluated on held-out US, China, and Japan data (January 2025) demonstrate cross-geography robustness of scale-invariant features.

Beyond temporal generalization, we assess zero-shot geographic transfer by evaluating TradeFM (trained exclusively on US equities) on held-out data from China and Japan. Despite substantial microstructural differences – including Japan’s Itayose batch auction mechanism (vs. US continuous trading), China’s $\pm 10\%$ daily price limits (vs. none in US), and wider bid-ask spreads in Asian markets (3–10 bps vs. 1–2 bps) – Figure 5 shows moderate perplexity degradation, with significant overlap between US and APAC distributions. This cross-market generalization demonstrates that scale-invariant order flow representations capture universal principles of market dynamics.

Controllability

Finally, we verify that the model respects its conditioning tokens and that its output can be reliably steered via the indicator features (I_{MP} and i_l). We generate 256 context-free trajectories of 512 tokens each, for every combination of market-participant and liquidity indicators. We then analyze the statistical properties of the raw generated orders by computing the standard deviation of volumes and interarrival times for each condition.

Figure 6 shows that modulating the indicator tokens has a significant and intuitive effect on the statistical properties of the generated orders, with two trends:

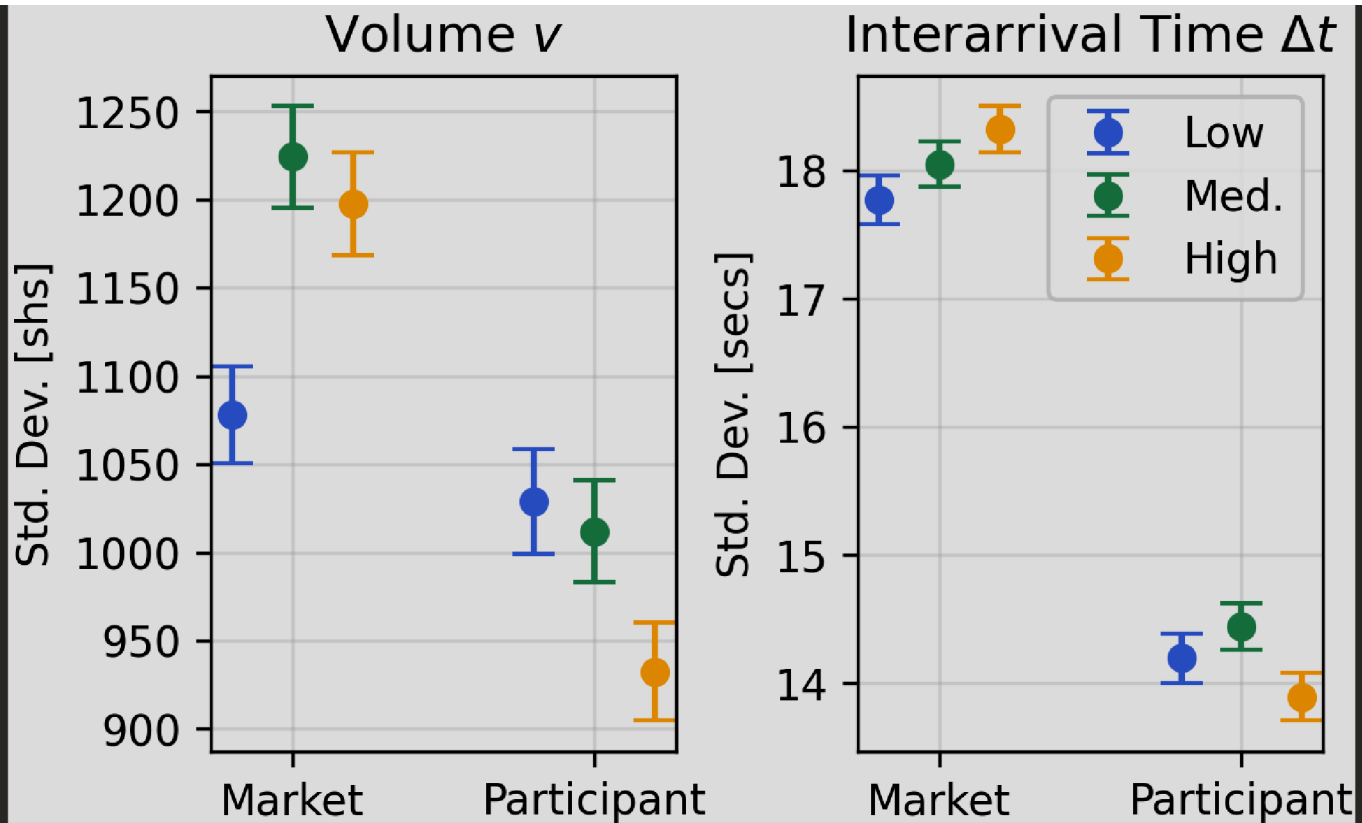


Figure 6: **Controllability experiments.** Standard deviation of generated volumes and interarrival times, conditioned on liquidity (i_l) and observation level (I_{MP}). The model produces distinct order flow, demonstrating controllable generation.

- The variance of both volume and interarrival time is consistently higher for market-level generation than for participant-level, aligned with the intuition that the aggregate behavior of an entire market is inherently more volatile than the behavior of a single participant.
- The model captures linear relationships between liquidity and the variance of interarrival times and volumes.

Collectively, these results demonstrate that TradeFM has learned a generalizable, conditional model of market behavior, capable of generating statistically and contextually appropriate order flow.

10 Conclusion

We have shown that the complex, emergent dynamics of financial markets can be learned directly from raw, heterogeneous order flow. Our end-to-end methodology, which combines scale-invariant feature representations with a universal tokenization scheme, allows a single generative Transformer to generalize across thousands of diverse assets without asset-specific calibration. The high fidelity of generated data – validated at both the emergent price dynamics level (Figure 4) and the granular order level (Figure 15, Appendix D.4) – combined with zero-shot geographic generalization suggests that scale-invariant trade representations capture fundamental, transferable structure in market microstructure. The closed-loop simulator opens several directions for future work: privacy-preserving synthetic data generation for illiquid assets, stress testing under counterfactual scenarios, and training learning-based trading agents via interaction with the simulator (preliminary explorations in Appendix D.4, D.5, D.6). Validating TradeFM's utility for these downstream applications remains a key priority.

Disclaimer

This paper was prepared for informational purposes by the Artificial Intelligence Research group of JPMorgan Chase & Co. and its affiliates ("JP Morgan") and is not a product of the Research Department of JP Morgan. JP Morgan makes no representation and warranty whatsoever and disclaims all liability, for the completeness, accuracy or reliability of the information contained herein. This document is not intended as investment research or investment advice, or a recommendation, offer or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction, and shall not constitute a solicitation under any jurisdiction or to any person, if such solicitation under such jurisdiction or to such person would be unlawful.

References

Bacry et al. (2015)

Bacry, E., Mastromatteo, I., and Muzy, J.-F. Hawkes processes in finance. *Market Microstructure and Liquidity*, 1(01):1550005, 2015.

Badaro et al. (2023)

Badaro, G., Saeed, M., and Papotti, P. Transformers for tabular data representation: A survey of models and applications.

Transactions of the Association for Computational Linguistics, 11:227–249, 2023.

Berkowitz et al. (1988)

Berkowitz, S. A., Logue, D. E., and Noser Jr., E. A.
The total cost of transactions on the NYSE.
The Journal of Finance, 43(1):97–112, 1988.

Berti & Kasneci (2025)

Berti, L. and Kasneci, G.
TLOB: A novel transformer model with dual attention for stock price trend prediction with limit order book data.
arXiv preprint arXiv:2502.15757, 2025.

Bouchaud (2010)

Bouchaud, J.-P.
The endogenous dynamics of markets: Price impact and feedback loops.
arXiv preprint arXiv:1009.2928, 2010.

Byrd et al. (2020)

Byrd, D., Hybinette, M., and Balch, T. H.
ABIDES: Towards high-fidelity multi-agent market simulation.
In *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*, pp. 11–22, 2020.

Cont (2001)

Cont, R.
Empirical properties of asset returns: Stylized facts and statistical issues.
Quantitative Finance, 1(2):223–236, 2001.

Dwarakanath et al. (2024)

Dwarakanath, K., Vyetrenko, S., Tavallali, P., and Balch, T.
ABIDES-Economist: Agent-based simulator of economic systems with learning agents.
arXiv preprint arXiv:2402.09563, 2024.

Farmer et al. (2005)

Farmer, J. D., Patelli, P., and Zovko, I. I.
The predictive power of zero intelligence in financial markets.

Proceedings of the National Academy of Sciences, 102(6):2254–2259, 2005.

Garza et al. (2023)

Garza, A., Challu, C., and Mergenthaler-Canseco, M.
TimeGPT-1.
arXiv preprint arXiv:2310.03589, 2023.

Gode & Sunder (1993)

Gode, D. K. and Sunder, S.
Allocative efficiency of markets with zero-intelligence traders: Market as a partial substitute for individual rationality.
Journal of Political Economy, 101(1):119–137, 1993.

Hasbrouck (2007)

Hasbrouck, J.
Empirical Market Microstructure: The Institutions, Economics, and Econometrics of Securities Trading.
Oxford University Press, 2007.

Hoffmann et al. (2022)

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., and Sifre, L.
Training compute-optimal large language models.
In *Advances in Neural Information Processing Systems*, volume 35, pp. 30016–30030, 2022.

Hollmann et al. (2025)

Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., Körfer, M., Hoo, S. B., Schirrmeyer, R. T., and Hutter, F.
Accurate predictions on small data with a tabular foundation model.
Nature, 637(8045):319–326, 2025.

Jain et al. (2024)	Jain, K., Firoozye, N., Kochems, J., and Treleaven, P. Limit order book dynamics and order size modelling using compound Hawkes process. <i>Finance Research Letters</i>, 69:106157, 2024.
Ji et al. (2021)	Ji, Y., Zhou, Z., Liu, H., and Davuluri, R. V. DNABERT: Pre-trained bidirectional encoder representations from transformers model for DNA-language in genome. <i>Bioinformatics</i>, 37(15):2112–2120, 2021.
Kaplan et al. (2020)	Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. <i>arXiv preprint arXiv:2001.08361</i>, 2020.
Kawawa-Beaudan et al. (2024)	Kawawa-Beaudan, M., Sood, S., Palande, S., Mani, G., Balch, T., and Veloso, M. Ensemble methods for sequence classification with hidden Markov models. <i>arXiv preprint arXiv:2409.07619</i>, 2024.
Li et al. (2025)	Li, J., Liu, Y., Liu, W., Fang, S., Wang, L., Xu, C., and Bian, J. MarS: A financial market simulation engine powered by generative foundation model. In <i>Proceedings of the Thirteenth International Conference on Learning Representations</i>, 2025.
Muennighoff et al. (2023)	Muennighoff, N., Rush, A. M., Barak, B., Le Scao, T., Piktus, A., Tazi, N., Pyysalo, S., Wolf, T., and Raffel, C. Scaling data-constrained language models. In <i>Advances in Neural Information Processing Systems</i>, volume 36, 2023.

Nagy et al. (2023)	Nagy, P., Frey, S., Sapora, S., Li, K., Calinescu, A., Zohren, S., and Foerster, J. Generative AI for end-to-end limit order book modelling: A token-level autoregressive generative model of message flow using a deep state space network. In <i>Proceedings of the Fourth ACM International Conference on AI in Finance</i>, pp. 91–99, 2023.
Nagy et al. (2025)	Nagy, P., Frey, S., Li, K., Sarkar, B., Vyetenko, S., Zohren, S., Calinescu, A., and Foerster, J. LOB-Bench: Benchmarking generative AI for finance – an application to limit order book data. In <i>Proceedings of the Forty-Second International Conference on Machine Learning</i>, 2025.
Nasdaq Listing Center (2024)	Nasdaq Listing Center. Equity trading rules. https://listingcenter.nasdaq.com/rulebook/nasdaq/rules/Nasdaq%20Equity%204, 2024.
Pasca (2015)	Pasca, L. A critical review of the main approaches on financial market dynamics modelling. <i>Journal of Heterodox Economics</i>, 2(2):151–167, 2015. doi: 10.1515/jheec-2015-0017.
Petty et al. (2024)	Petty, J., van Steenkiste, S., Dasgupta, I., Sha, F., Garrette, D., and Linzen, T. The impact of depth on compositional generalization in transformer language models.

In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 7239–7252. Association for Computational Linguistics, 2024.

Potluru et al. (2024)

Potluru, V. K., Borrajo, D., Coletta, A., Dalmaso, N., El-Laham, Y., Fons, E., Ghassemi, M., Gopalakrishnan, S., Gosai, V., Kreačić, E., Mani, G., Obitayo, S., Paramanand, D., Raman, N., Solonin, M., Sood, S., Vyetenko, S., Zhu, H., Veloso, M., and Balch, T.
Synthetic data applications in finance.
arXiv preprint arXiv:2401.00081, 2024.

Rajbhandari et al. (2020)

Rajbhandari, S., Rasley, J., Ruwase, O., and He, Y.
ZeRO: Memory optimizations toward training trillion parameter models.
In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–16. IEEE, 2020.

Raman et al. (2024)

Raman, N., Ganesh, S., and Veloso, M.
Scalable representation learning for multimodal tabular transactions.
arXiv preprint arXiv:2410.07851, 2024.

Ratliff-Crain et al. (2025)

Ratliff-Crain, E., Van Oort, C. M., Koehler, M. T. K., and Tivnan, B. F.
Revisiting Cont’s stylized facts for modern stock markets.
Quantitative Finance, 25(9):1343–1373, 2025.

Sirignano & Cont (2021)

Sirignano, J. and Cont, R.
Universal features of price formation in financial markets: Perspectives from deep learning.

In *Machine Learning and AI in Finance*, pp. 5–15. Routledge, 2021.

Touvron et al. (2023)

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G.
LLaMA: Open and efficient foundation language models.
arXiv preprint arXiv:2302.13971, 2023.

Vaswani et al. (2017)

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I.
Attention is all you need.
In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Vyetrenko et al. (2020)

Vyetrenko, S., Byrd, D., Petosa, N., Mahfouz, M., Dervovic, D., Veloso, M., and Balch, T. H.
Get real: Realism metrics for robust limit order book market simulations.
In *Proceedings of the First ACM International Conference on AI in Finance*, pp. 1–8, 2020.

Wen et al. (2023)

Wen, Q., Zhou, T., Zhang, C., Chen, W., Ma, Z., Yan, J., and Sun, L.
Transformers in time series: A survey.
In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, pp. 6778–6786, 2023.

Wiese et al. (2020)

Wiese, M., Knobloch, R., Korn, R., and Kretschmer, P.
Quant GANs: Deep generation of financial time series.
Quantitative Finance, 20(9):1419–1440, 2020.

Xiao et al. (2025)

Xiao, Y., Ventre, C., Wang, Y., Li, H., Huan, Y., and Liu, B.

LiT: Limit order book transformer.
Frontiers in Artificial Intelligence,
8:1616485, 2025.

Zhang et al. (2019)

Zhang, Z., Zohren, S., and Roberts, S.
DeepLOB: Deep convolutional neural
networks for limit order books.
IEEE Transactions on Signal Processing,
67(11):3001–3012, 2019.

Appendix A Appendix

A.1 Foundation Models for Structured Data

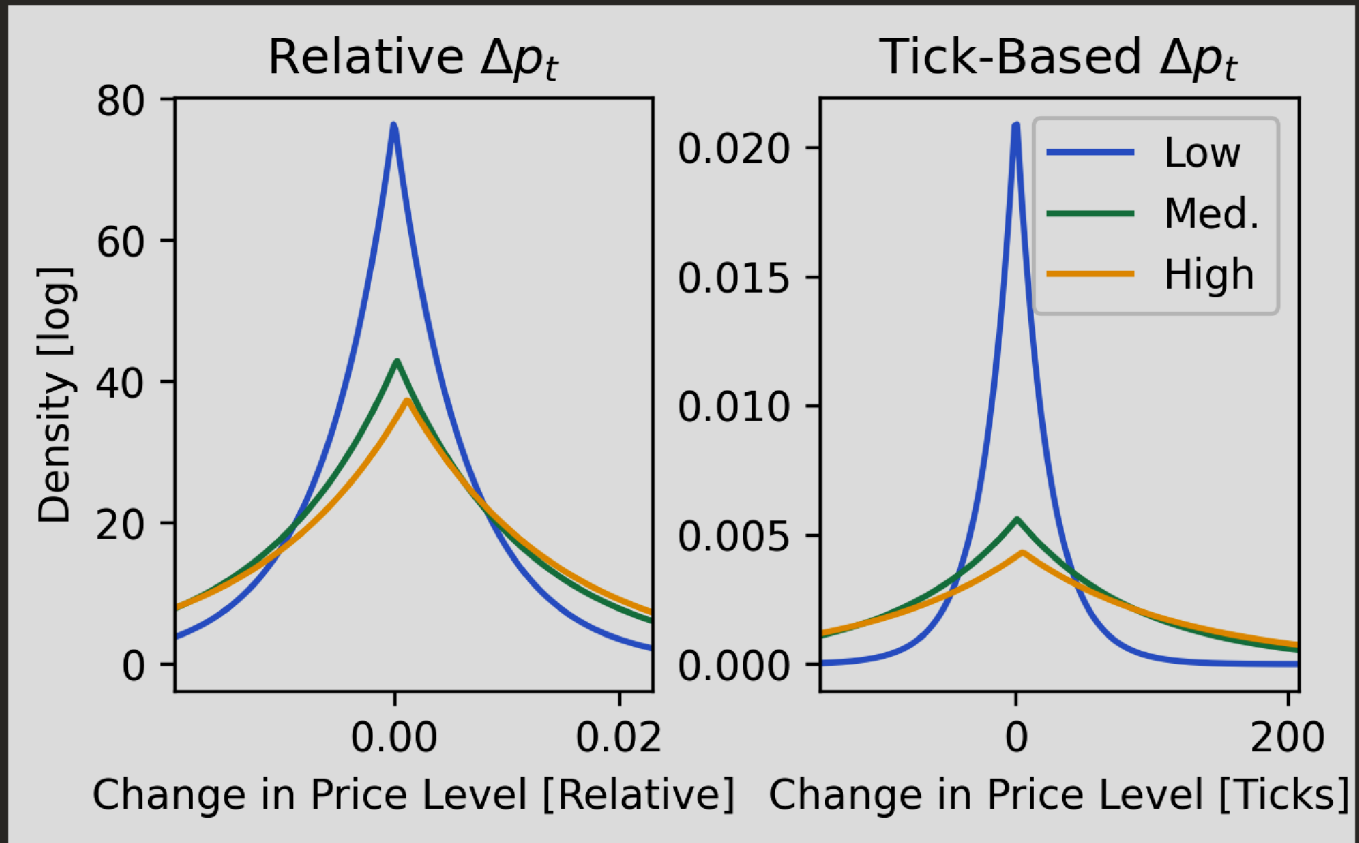


Figure 7: **Tick-based vs. relative features.** Properties of tick-based vs. relative feature construction for the sample feature Δp_t , across liquidity profiles. Relative features generalize better across assets than absolute, tick-based features.

This work draws inspiration from the success of foundation models beyond NLP, in domains with structured sequential data. This paradigm has been successfully adapted by treating domain-specific sequences as a form of "language." In genomics, for example, models treat DNA or protein sequences as sentences to learn fundamental biological patterns (Ji et al., 2021). For general-purpose time-series forecasting, large pre-trained models have demonstrated strong zero-shot performance on unseen series (Garza et al., 2023). Similarly, for tabular data, Transformers pre-trained on a diverse collection of tables can perform inference on new, smaller tables without task-specific fine-tuning (Badaro et al., 2023; Hollmann et al., 2025). In finance, by framing order flow as a structured language of market events, our approach aligns with this proven paradigm, arguing for its direct applicability to the unique challenges of financial data.

A.2 The Transformer as a Natural Fit

The Transformer architecture is uniquely suited to address these challenges. Its core components map naturally to the fundamental properties of order flow data:

- **Self-Attention:** The attention mechanism is designed to capture complex, long-range dependencies within a sequence. This makes it an ideal tool for modeling the long memory and intricate, non-linear interactions inherent in order flow.
- **Sequence-to-Sequence Framework:** As an autoregressive, sequence-based model, the Transformer inherently handles the asynchronous, event-driven nature of the data, where the time between events is itself a feature to be learned.
- **Adapting to Multi-feature Sequences:** While Transformers excel at processing univariate text, our trade events are multi-feature tuples. A key challenge is thus to effectively discretize and tokenize these mixed-type features into a processable sequence, which motivates our novel tokenization and embedding methodology.

We develop an end-to-end methodology to build TradeFM, a generative foundation model for market microstructure. The following four sections detail each component of our pipeline: our data processing and scale-invariant feature engineering (Section 5), our universal tokenization scheme (Section 6), the TradeFM model architecture (Section 7), and the closed-loop market simulator used for evaluation (Section 8).

A.3 Mid-Price Estimation

A robust estimate of the true market mid-price (p_t^{mid}) is critical for normalizing price-related features. While dedicated market data sources for this exist, they are often expensive. Given our access to raw transaction data, we seek to estimate this value directly. In our partial-information setting, we primarily observe the execution prices (p_t^{exec}) for consummated trades. The raw stream of p_t^{exec} is a noisy version of the true mid-price p_t^{mid} .

A naive approach, such as a simple rolling average of execution prices, is insufficient. A fixed-width window of trades is not comparable across assets with different liquidity levels; a 50-trade window may span less than a second for a highly liquid asset but several hours for an illiquid one. A time-based window (e.g., 2 seconds) is more relevant, but a simple average still fails to account for trade volume. For example, an average that gives equal weight to a 1,000-share trade at \$10.00 and a 1-share trade at \$9.00 would produce a misleading estimate.

The conventional solution to this problem is the volume-weighted average price (VWAP), which is

$$\hat{p}_t^{\text{VWAP}} = \frac{\sum_{i=0}^W v_{t-i} p_{t-i}^{\text{exec}}}{\sum_{i=0}^W v_{t-i}} \quad (2)$$

To make this estimator more reactive to recent information, we introduce **Exponentially-Weighted Volume-Weighted Average Price (EW-VWAP)**. This is calculated by maintaining two separate exponential moving averages (EMAs): one for the volume-weighted price and one for the volume itself. For each incoming trade with execution price p_t^{exec} and volume v_t , we update the EMAs for the numerator (N_t) and denominator (D_t) as follows:

$$N_t = \alpha \cdot (p_t^{\text{exec}} \cdot v_t) + (1 - \alpha) \cdot N_{t-1}$$

$$D_t = \alpha \cdot v_t + (1 - \alpha) \cdot D_{t-1}$$

The EW-VWAP at time t is then the ratio of these two values:

$$\hat{p}_t^{\text{EW-VWAP}} = \frac{N_t}{D_t} \quad (3)$$

The smoothing factor α is determined by a time-based halflife, ensuring that the estimate gives more weight to larger and more recent trades in a temporally consistent manner. This provides a stable and representative price benchmark that reflects the price at which the bulk of recent market activity has occurred. This estimator is used throughout the paper to normalize all price-related features to a common scale.

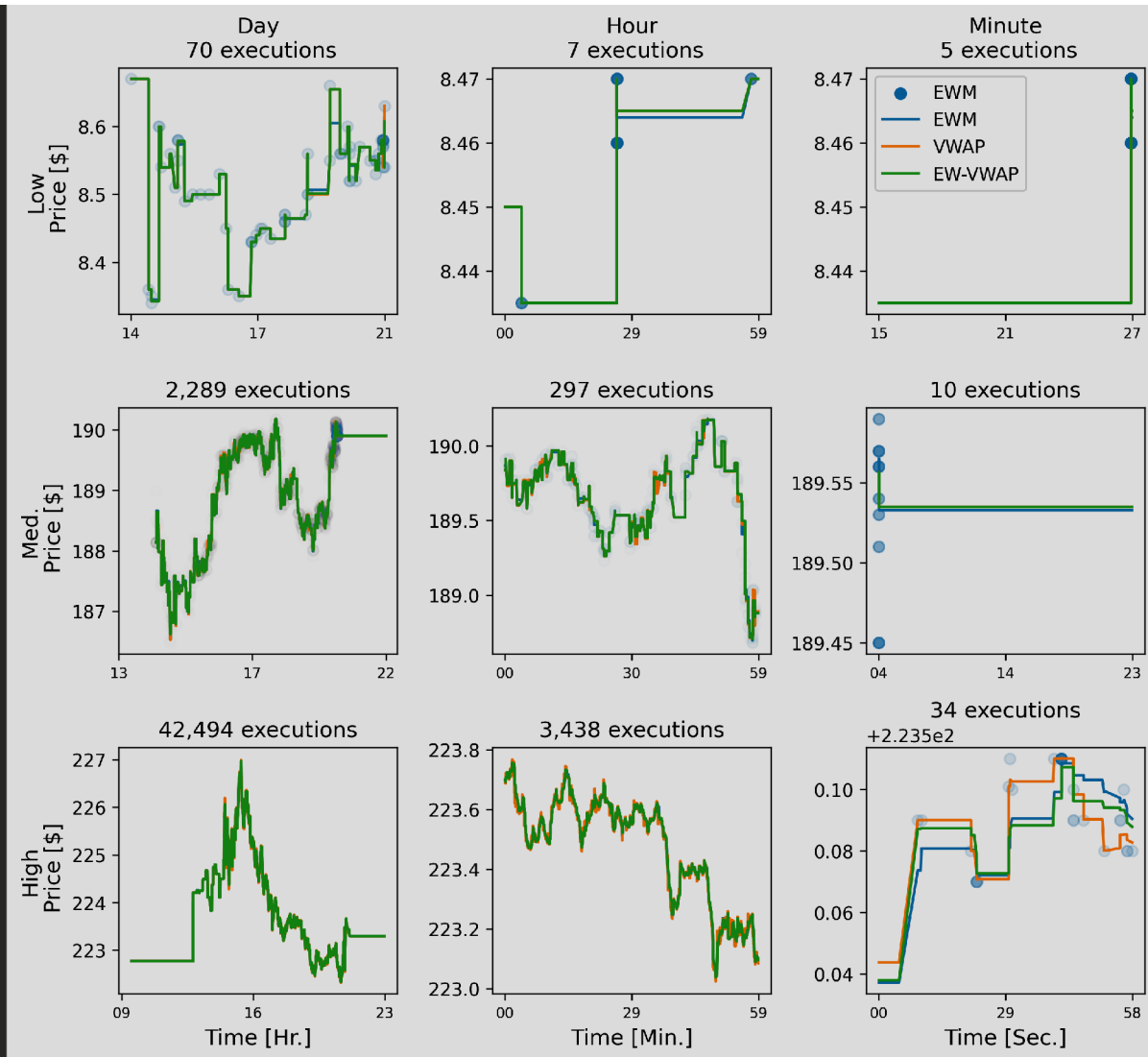


Figure 8: **Mid-price estimator comparison.** Our proposed EW-VWAP provides a more stable and responsive estimate than standard VWAP or EWM, closely tracking the executed fill prices across different time scales and liquidity regimes.

A.4 Tokenization Example

Time-step	Time-stamp	Asset	Avg. Daily Vol. (shs)	Midprice (\$)	Action	Side	Order Price (\$)	Vol. (shs)
				⋮				
42	09:45:30	AAPL	53,496,022	182.45	ADD	BUY	182.44	500
43	09:45:38	AAPL	53,496,022	182.48	ADD	SELL	182.50	750
44	09:45:52	AAPL	53,496,022	182.50	CANCEL	BUY	182.49	300
				⋮				

Table 4: **Example trade sequence.** Toy example of trading activity for an imaginary AAPL sequence, demonstrating the multi-feature and heterogeneous nature of our data pre-tokenization.

Given the imaginary sequence of trade events e_t constructed in Table 4, our features for timestep $t = 43$ are as follows:

- $\Delta t_t = w_t - w_{t-1} = 09:45:38 - 09:45:30 = 8\text{sec}$
- $\delta p_t = \frac{p_t^{\text{order}} - \hat{p}_t^{\text{mid}}}{\hat{p}_t^{\text{mid}}} = \frac{\$182.50 - \$182.48}{\$182.48} = \frac{\$0.02}{\$182.48} = 0.011\% = +1.1 \text{ bps}$
- $v_t = 750\text{shs}$
- $a_t = \text{Add Order}$
- $s_t = \text{Sell}$

Using our calibrated bins, we would discretize these features to the bin indices:

- $i_{\Delta t_t} = \text{bin } 11$
- $i_{\delta p_t} = \text{bin } 7$
- $i_{v_t} = \text{bin } 7$
- $i_{a_t} = \text{bin } 0$
- $i_{s_t} = \text{bin } 1$

Using Equation 1 would yield:

$$i_{\text{trade}} = 6011$$

Assuming an opening price $p_0 = \$179.50$, we would have change in price level feature:

$$\Delta p_t = \frac{\hat{p}_t^{\text{mid}} - p_0}{p_0} = \frac{\$182.48 - \$179.50}{\$179.50} = 1.66\% = +166 \text{ bps}$$

Discretizing this using our bins would give $i_{\Delta p_t} = 19$. Based on the asset’s average daily volume of 53 million, which falls into the high liquidity bin, our liquidity indicator i_l would be 2. If this sequence was a market-level sequence, we would have market-participant indicator $I_{MP} = 0$.

Our final model input would then be:

$$[i_l, I_{MP}, i_{\Delta p_t}, i_{\text{trade}}] = [2, 0, 19, 6011].$$

Model Size	Num. Train Tokens	Batch Size	GPUs	Train Time / Epoch (hrs)	Optimization
125M	2.6B	24	3xA100	17	Accelerate
250M	6.4B	32	4xA10G	29	DeepSpeed (Rajbhandari et al., 2020)
500M	10.7B	24	3xA100	128	Accelerate

Table 5: **Training configuration.** Setup details for different model sizes, including token counts, batch sizes, hardware, and training time per epoch.

Appendix B Reproducibility Guide

B.1 Model Backbone

TradeFM is a decoder-only Transformer, trained from scratch with a custom configuration. The architecture is based on the Llama family (Touvron et al., [2023](#)) and incorporates modern enhancements for efficiency and performance, including:

- **Grouped-Query Attention (GQA):** Balances the performance of Multi-Head Attention with the reduced memory bandwidth of Multi-Query Attention, enabling faster inference and larger batch sizes.

- **Rotary Position Embeddings (RoPE)**: Encodes relative positional information by applying a rotation to query and key vectors, which has been shown to improve generalization for long sequences.

B.2 Model Hyperparameters and Scaling

The model size is guided by the Chinchilla scaling laws, which prescribe a compute-optimal balance between model size and unique training tokens, implying approximately 20 tokens per parameter (Kaplan et al., [2020](#); Hoffmann et al., [2022](#)). Given our dataset of 10.7 billion unique tokens, this implies a target model size of around 525 million parameters. We note that our 4-epoch training schedule means the model sees approximately 42.8B total tokens, exceeding this ratio with repeated data; this follows recommendations for training in data-constrained regimes (Muennighoff et al., [2023](#)). Our final hyperparameters are as follows:

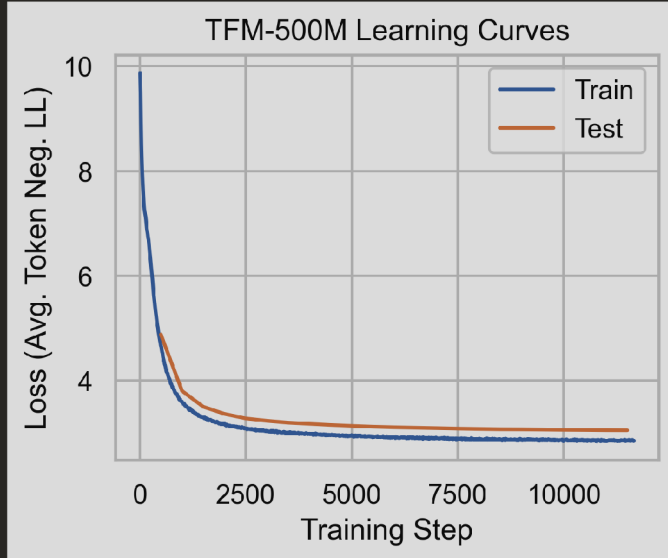
- **Context Length**: 1,024 tokens
- **Hidden Layers**: 32
- **Embedding Dimension**: 1,024
- **Intermediate MLP Size**: 4,096
- **Attention Heads**: 32
- **Key-Value Heads (GQA)**: 8 heads, 4 groups
- **Total Parameters**: 524.4 Million

The 1:4 ratio between the embedding dimension and the intermediate MLP size is chosen in accordance with best practices for Transformer models (Petty et al., [2024](#)).

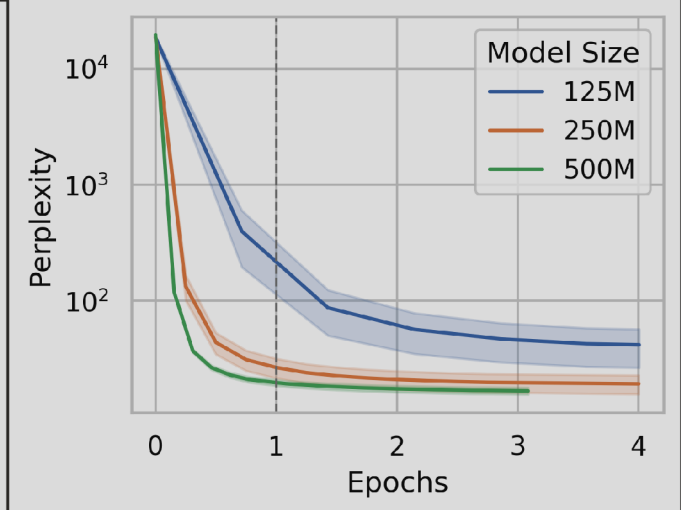
B.3 Training Configuration

We train the model on an AWS instance with 3 Nvidia A100 GPUs, each with 80GB of RAM. All training is performed in **fp16** half-precision. To achieve an effective batch size of 4,032, we use a per-device batch size of 24 and gradient accumulation over 56 steps. For further memory optimization and training acceleration, we use the Accelerate library. The model is trained using the AdamW optimizer with a linear learning rate schedule, a learning rate of 5×10^{-5} , and 500 steps of warmup. Following recommendations for training on large datasets (Muennighoff et al., [2023](#)), we train for a total of 4 epochs.

Due to compute constraints, different model sizes in memory, and different dataset sizes implied by Chinchilla scaling laws, our training setups vary slightly between model sizes. We summarize these variations in Table 5.



(a) **TradeFM-500M learning curve.** Average token negative log likelihood over the test set constituting Jan. through Sept. 2025.



(b) **Perplexity vs. epochs.** Out-of-sample perplexity for all TradeFM models. Epoch 1 represents one pass over the entire dataset; subsequent training is on repeated data.

Figure 9: **Training dynamics.** (a) Learning curve for TradeFM-500M. (b) Out-of-sample perplexity across model sizes and epochs.

B.4 Tokenizer Pseudocode

Algorithm 1 Tokenizer Pseudocode

```
0: | Flattened trading data with features: price, volume, time, action, side, participant
0: | Bin counts:  $N_{\text{price}}, N_{\text{depth}}, N_{\text{volume}}, N_{\text{time}}, N_{\text{side}} = 2, N_{\text{action}} = 2$ 
1: | for each feature  $f$  in {price, depth, volume, time} do
2: |   Remove NaN and infinite values from  $f$ 
3: |   Compute upper outlier threshold  $u = \text{percentile}(f, 99)$ 
4: |   if feature is double-sided then
5: |     Compute lower outlier threshold  $l = \text{percentile}(f, 1)$ 
6: |     Assign values outside  $[l, u]$  to lower / upper outlier bins
7: |   else
8: |     Assign values above  $u$  to upper outlier bin
9: |   end if
10: | if using equal-frequency bins then
11: |   Compute bin edges  $B_f$  using quantile binning with  $N_f$  bins
12: | else
13: |   Compute bin edges  $B_f$  using histogram binning with  $N_f$  bins
14: | end if
15: | Digitize  $f$  into bin indices  $I_f$  using  $B_f$ 
16: | Handle outliers: assign out-of-range values to edge bins as needed
17: | end for
18: | for each categorical feature  $c$  in {action, side, participant} do
19: |   Map each category to a unique integer index  $I_c$ 
20: | end for
21: | for each order  $o$  in the dataset do
22: |   for each feature  $f$  do
23: |     if  $o[f]$  is NaN then
24: |       Impute  $o[f]$  with a random valid bin index or default value
25: |     end if
26: |   end for
27: |   Compute token index for  $o$ :
28: |    $T_o = I_{\text{action}} \times N_{\text{side}} \times N_{\text{depth}} \times N_{\text{volume}} \times N_{\text{time}}$ 
29: |    $+ I_{\text{side}} \times N_{\text{depth}} \times N_{\text{volume}} \times N_{\text{time}}$ 
30: |    $+ I_{\text{depth}} \times N_{\text{volume}} \times N_{\text{time}}$ 
31: |    $+ I_{\text{volume}} \times N_{\text{time}}$ 
32: |    $+ I_{\text{time}}$ 
33: |   Assign  $T_o$  to order  $o$ 
34: | end for
```

B.5 Market Simulation Pseudocode

Algorithm 2 Market Simulator: Part 1 - Initialization and Utilities

```
0: | Sequence of order transactions, initial price  $p_0$ , simulation parameters
1: | Initialize Exchange:
2: | Set initial price  $p_0$ 
3: | Initialize order book, midprice, fills, deletes, spreads, bid/ask volumes
4: | Function: INITIALIZEORDERBOOK(order_columns)
5: | Reset order book, midprice, fills, deletes, spreads, bid/ask volumes
6: | Set initial bid/ask to  $p_0$ 
7: | Function: GETORDERPRICE(transaction)
8: | if order is market then
9: |   if side is Sell then
10: |     price  $\leftarrow$  lowest ask
11: |   else
12: |     price  $\leftarrow$  highest bid
13: |   end if
14: | else
15: |   price  $\leftarrow$  (order price depth / 10,000)  $\times$  current midprice + current midprice
16: | end if
17: | Return price
18: | Function: GENERATEFILL(best_past_order, order, quantity)
19: | Compute time since best_past_order
20: | Determine match price:
21: | if both orders are market then
22: |   price  $\leftarrow$  current midprice
23: | else if order is market then
24: |   price  $\leftarrow$  best_past_order price
25: | else if best_past_order is market then
26: |   price  $\leftarrow$  order price
27: | else
28: |   price  $\leftarrow$  best_past_order price
29: | end if
30: | Return fill record with IDs, sides, prices, depths, volume, time delta
```

Algorithm 3 Market Simulator: Part 2 - Simulation Step Functions

Function: STEPORDERBOOK(order)

Extract side and price from order

while order volume > 0 **do**

Find matching opposite-side orders based on price-time priority

if no matching orders **then**

break

end if

Select best matching order (highest bid or lowest ask)

if best_past_order volume > order volume **then**

Reduce best_past_order volume by order volume

Record fill and **return**

else if best_past_order volume < order volume **then**

Reduce order volume by best_past_order volume

Remove best_past_order from book

Record fill

else

Record fill

Remove best_past_order from book

return

end if

end while

if order volume > 0 **then**

Add partially filled order to book

end if

Function: STEPMIDPRICE(transaction)

if transaction is Delete **then**

Use current order book

else

Add transaction to temporary order book

end if

Update highest bid and lowest ask from book

Compute midprice as average of highest bid and lowest ask

Record midprice and bid/ask volumes

Function: STEPSIM(transaction)

Update transaction midprice

if action is Add **then**

Compute order price

Update midprice

Step order book

else if action is Delete **then**

Match on order ID

Remove matching orders and record deletes

Update midprice

end if

Record simulation time for profiling

Function: RUNSIMULATION(data)

Initialize order book

for each transaction in data **do**

StepSim(transaction)

end for

Return fills and midprice history

B.6 Zero-Intelligence Baseline

The Zero-Intelligence (ZI) agent is a canonical null model used to test whether a model learns complex, conditional dynamics beyond the market's basic structural properties (Gode & Sunder, [1993](#); Farmer et al., [2005](#)). To provide a fair baseline, our ZI agent generates orders stochastically by sampling from distributions calibrated to match the marginals of key features in a 450-million-trade sample of the training data.

Specifically, side and action type are sampled from their empirical categorical distributions; interarrival time is sampled from a fitted Exponential distribution; order volume from a fitted Exponential distribution; and price depth is drawn from a Gaussian Mixture Model (GMM).

The resulting ZI agent orders are processed through the identical market simulator and evaluation pipeline as TradeFM to ensure a direct and fair comparison. We compute 2,048 rollouts of 1,024 events, and compute the same stylized facts.

B.7 Compound Hawkes Baseline

Hawkes Processes are commonly applied to market data for their ability to robustly model interarrival times of self-exciting events (Bacry et al., [2015](#); Jain et al., [2024](#)). We adopt the Compound Hawkes model which combines a Hawkes process for modeling interarrival times with empirical distributions for modeling additional event features like volume and price depth. We use the same 450-million-trade data as is used to train our zero-intelligence baseline, and separate the data based on action and side.

We then fit a Hawkes process using a sum of exponential kernel, with 4 dimensions, one for each combination of action and side (buy-delete, buy-add, sell-delete, sell-add). For each of these action-side combinations we calibrate a Gaussian Mixture Model for price depths, and an Exponential for volume.

Appendix C Scaling Analysis

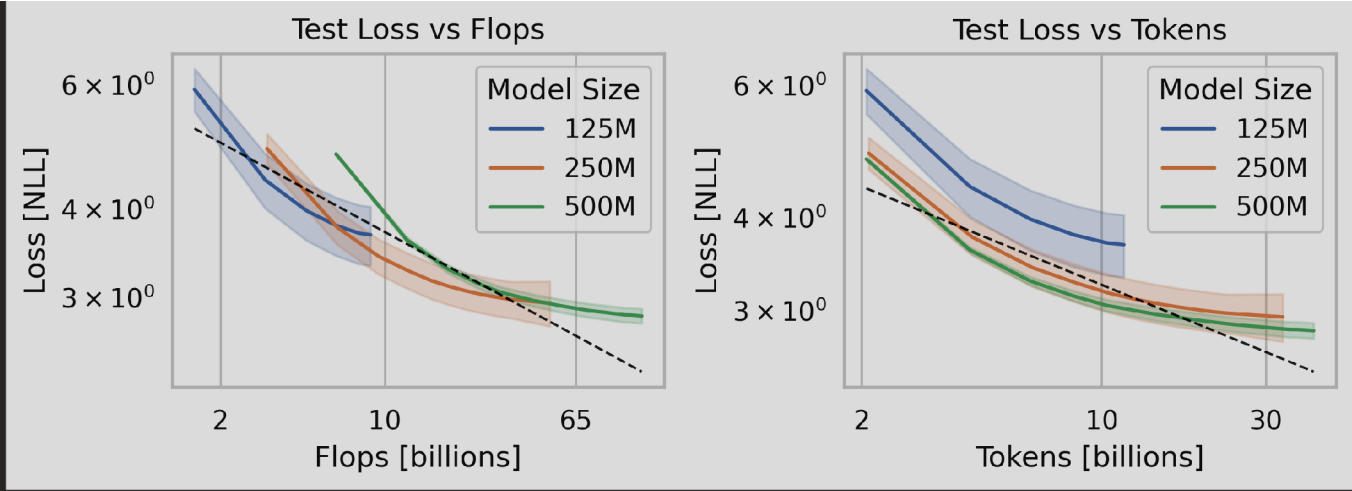


Figure 10: **Scaling law analysis.** Test loss (negative log likelihood) on held-out data one month in advance of the training data cutoff. The black dashed line represents the power law fit to the minimum loss frontier.

To substantiate the *Foundation Model* claim, we conducted a comprehensive scaling analysis of our approach. We trained models at three scales: approximately 125M, 250M, and 500M parameters.

Our **500M parameter model** is our largest model and serves as the primary evaluation target throughout this paper. The scaling law plots in Figure 10 demonstrate expected power-law relationships between compute, dataset size, and test loss. These plots include repeated data, as we train for four epochs. We verify this by computing the minimum loss frontier for compute and dataset size, fitting power laws to find that the test loss $L(C)$ with respect to compute in Flops C , and $L(D)$ with respect to dataset size in tokens D , follow:

$$L(C) \propto C^{-\alpha_C}, \quad \alpha_C \approx 0.18$$

$$L(D) \propto D^{-\alpha_D}, \quad \alpha_D \approx 0.19.$$

While 500M is small relative to general-purpose LLMs, it is large for the Financial Microstructure domain, and comparable in scale to other domain-specific models such as MaRS (~ 1 B parameters). Standard SOTA models in this field typically have < 10 M parameters (e.g., DeepLOB (60K params) and LOBS5 (6.3M params)). TradeFM represents a **$> 50 \times$ increase in model capacity** over existing domain-specific baselines.

Appendix D Extended Experimental Results

D.1 Geographic Zero-Shot Generalization

We evaluate the model, trained exclusively on US equities, on a hold-out set of assets from APAC markets (China and Japan). We detail these held-out datasets in Table 6. Figure 5 shows the distribution of perplexity scores for one month of data (held out for all geographies), with significant overlap between US and APAC. The minimal degradation in perplexity on unseen markets confirms TradeFM’s generalization capabilities.

Country	Number of Assets	Date-Asset Pairs	Tokens
US	6,885	81,203	857,017,219
China	4,926	68,925	37,408,529
Japan	2,932	37,235	286,476,052

Table 6: **Geographic held-out datasets.** Dataset statistics for US, China, and Japan. All geographies are evaluated on Jan. 2025 data.

D.2 Simulator Validation

In order to evaluate the simulator, we replay sequences of real orders through it and compare the statistical properties of the resulting simulated trade fills against the real fills from our historical data. We focus on two key metrics: the cumulative distribution function (CDF) of fill volumes and the CDF of lot counts (the number of discrete fills required to complete a single order). As shown in Figure 11, we find a strong correspondence between the real and simulated distributions across assets of varying liquidity, confirming that our simulator is a high-fidelity environment for evaluation. We find correlations of 0.91 and 0.98 between sim and real volumes and lot counts, respectively.

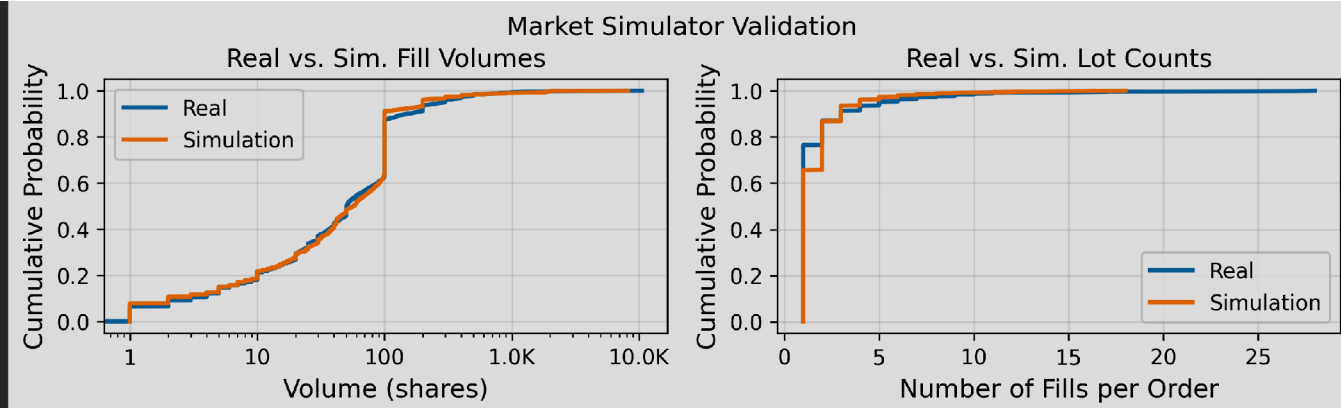


Figure 11: **Simulator fill validation.** Stylized facts of market simulator fills: (left) fill volumes; (right) lot counts. Correlations between simulated and actual fills are 0.91 for volumes and 0.98 for lot counts.

D.3 Temporal Drift

As financial markets are dynamic and market regimes are constantly changing, we investigate the tendency of model performance to drift over time. Our tokenizer's main contribution is to standardize representations of market features over both the liquidity and time regime.

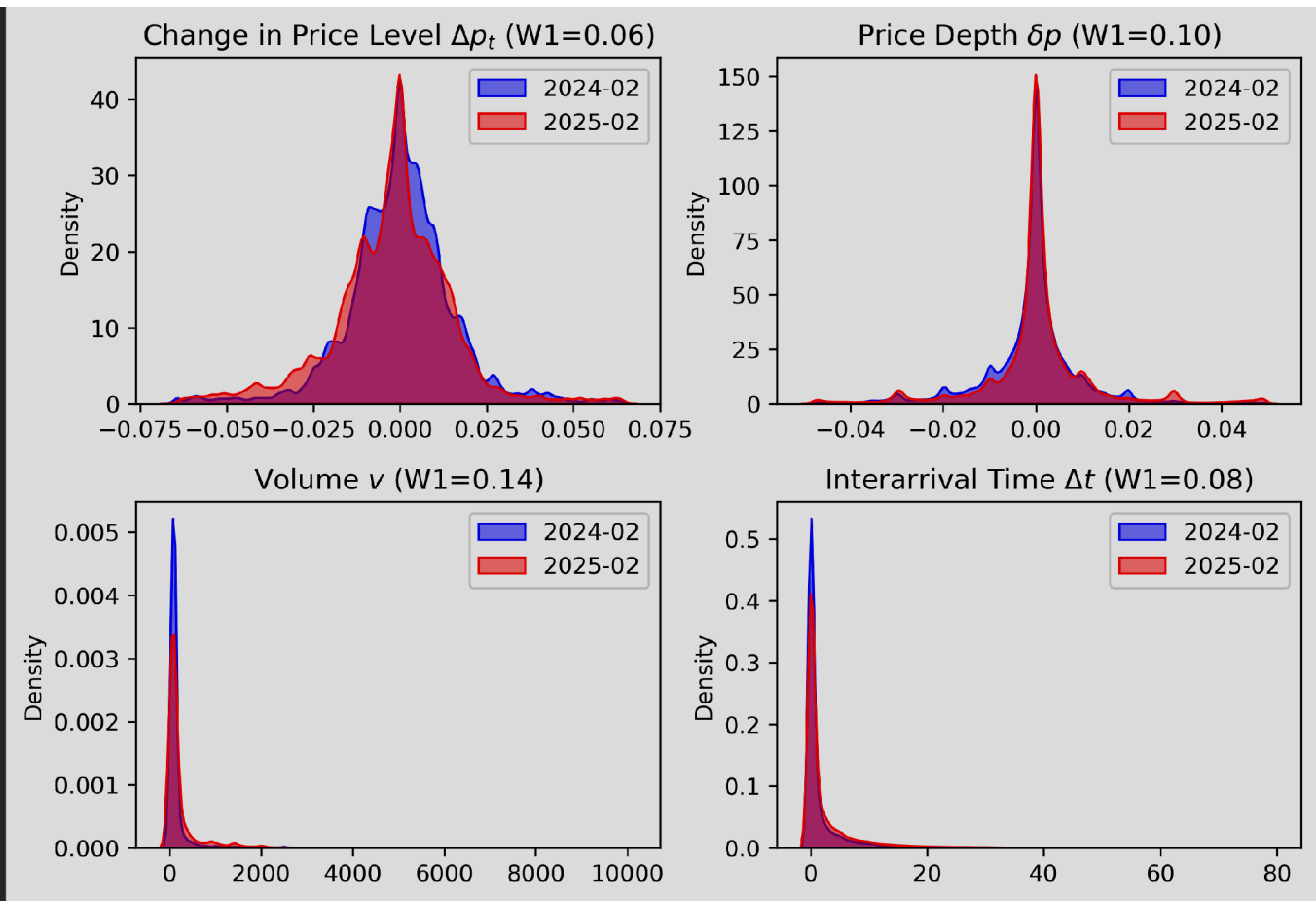


Figure 12: **Feature stationarity over time.** Kernel-density estimation of feature distributions from the tokenizer calibration period of Feb. 2024 to one year later in Feb. 2025. Our feature engineering successfully makes these features stationary, enabling generalization to out-of-distribution temporal regimes.

In Figure 12 we demonstrate the universality of these features by exploring the distribution of our relative price level, relative price depth, interarrival time, and volume features in both the month used to calibrate our tokenizer, Feb. 2024, and one year later in Feb. 2025. We observe that our features are stationary over this period even as volatility, price level, and other market conditions vary. Figure 13 shows the Kolmogorov-Smirnov and Wasserstein distance of each of these features between the tokenizer calibration month and each of 9 held-out months. We include a non-stationary feature, raw mid-price, to contextualize the stationarity of these metrics.

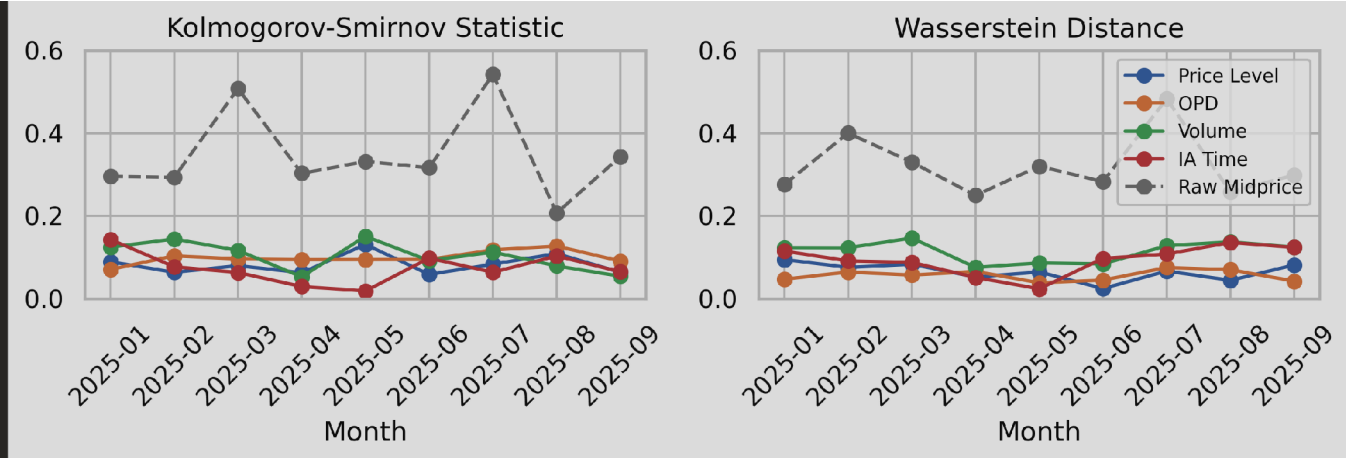


Figure 13: **Tokenizer drift over months.** Kolmogorov-Smirnov and Wasserstein distances between feature distributions during the tokenizer calibration month and held-out months. Raw mid-price, a non-stationary feature, is included for context. (OPD and IA Time in the legend correspond to Price Depth and Interarrival Time, respectively.)

In Figure 14 we extend the aggregated results in Table 3 for all quantities of interest. We observe that while these metrics do vary within a range, the variance is small and our method mostly achieves higher fidelity than baselines.

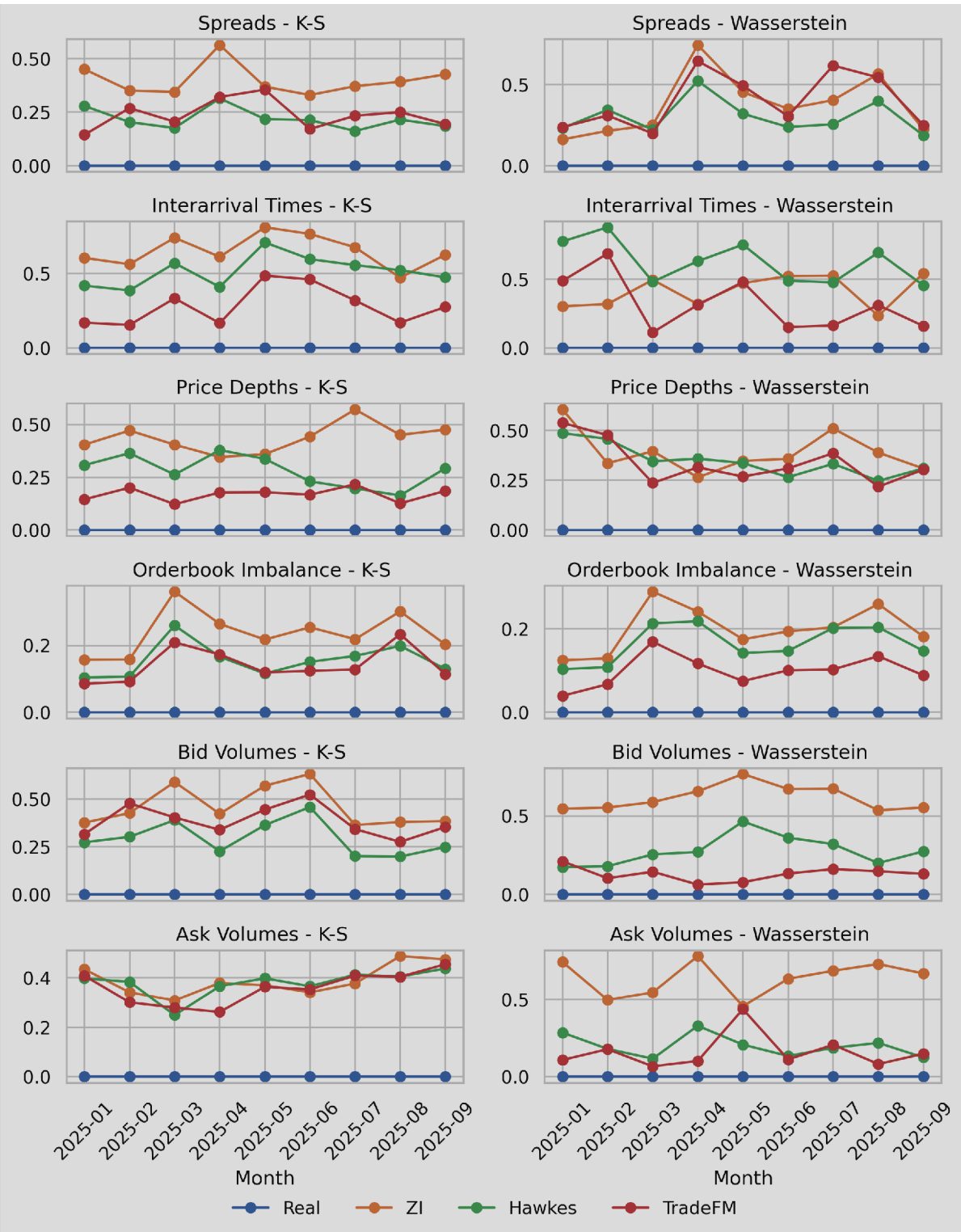


Figure 14: **Distributional fidelity over time.** Wasserstein distance and Kolmogorov-Smirnov statistic of feature distributions and emergent market factors from various methods over nine held-out months.

D.4 Synthetic Data Generation

TradeFM can serve as an engine for generating high-fidelity, privacy-preserving synthetic financial data. The realism of the generated data is validated not only at the level of emergent price dynamics (Figure 4), but also at the granular level of individual orders. As shown in Figure 15, the distributions of simulated order volumes and interarrival times closely match the power-law and exponential distributions observed in real data, respectively. This high-fidelity generation is valuable for:

- **Backtesting Trading Strategies:** Synthetic data allows for robust testing of trading algorithms against a wide range of plausible market scenarios, reducing the risk of overfitting to a single historical path (Potluru et al., 2024).
- **Augmenting Sparse Datasets:** For illiquid assets where historical data is sparse, the model can generate additional data to facilitate more robust analysis and model training.
- **Privacy-Preserving Data Sharing:** The model can generate realistic datasets for academic or public use without revealing sensitive, proprietary transaction information.

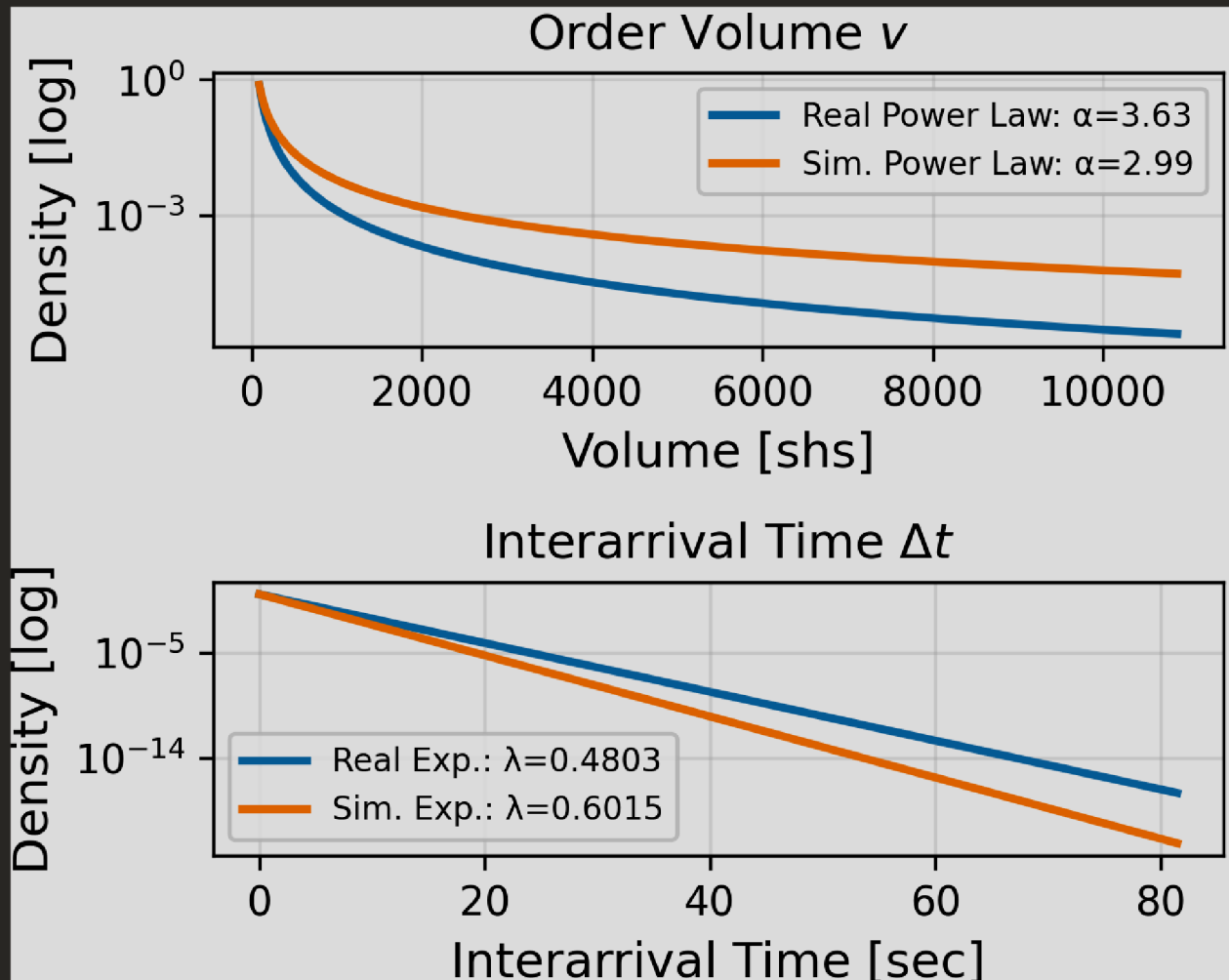


Figure 15: **Order-level distributional fidelity.** (Top) Simulated order volumes follow the power-law observed in real data. (Bottom) Simulated interarrival times match the exponential distribution of real data.

D.5 Market Simulation & Stress Testing

The integrated TradeFM-simulator system functions as a high-fidelity environment for complex “what-if” analyses and stress testing. This allows for the study of systemic risk and market stability in a controlled environment. The ability to generate plausible, multi-step forecasts of future market trajectories, as illustrated in Figure 16, is a direct outcome of this closed-loop simulation capability.

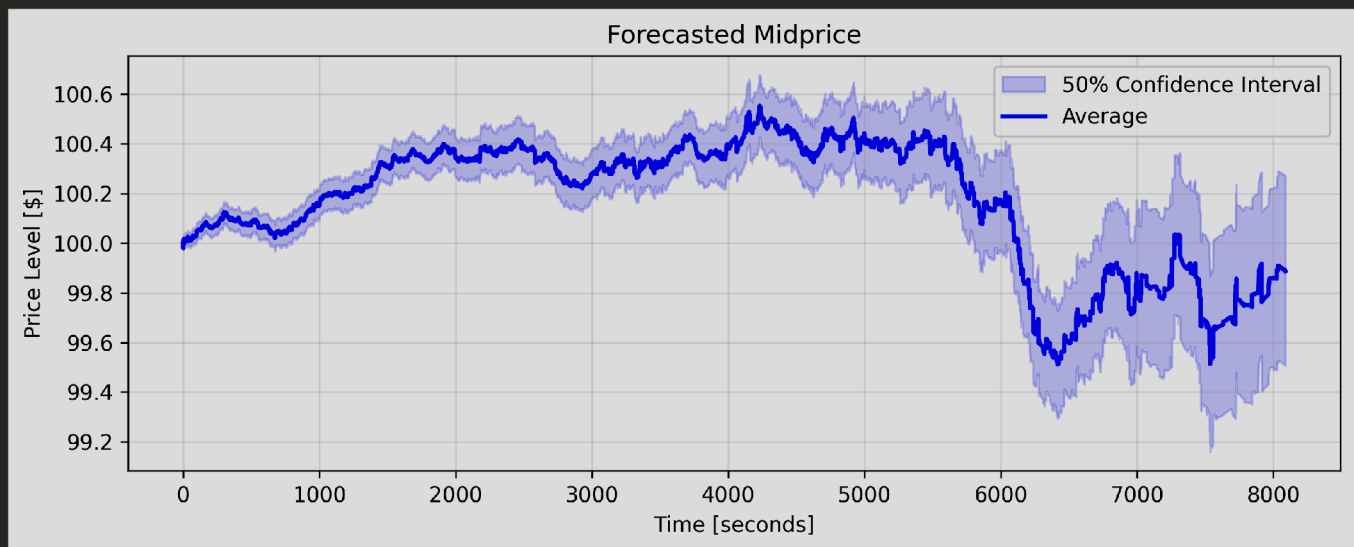


Figure 16: **Multi-step mid-price forecast.** Rollout-based forecast for an imaginary asset initially priced at \$100. The average trajectory and 50% confidence interval over 256 simulations show the model generates plausible, non-stationary market paths.

Such systems are also useful to regulators and risk managers (Dwarakanath et al., 2024), who can use this system to simulate the market’s response to extreme or counterfactual scenarios, such as by injecting large, anomalous orders into a historical context and observing the resulting price trajectory. Figure 17 demonstrates this capability – for a sample asset, we artificially inject buy or sell orders at 10x the frequency found in the real context, and average mid-price

forecasts over 10 rollouts. When we inject artificial sell orders, the mid-price drops, and when we inject buy orders, the mid-price rises, illustrating realistic behavior.

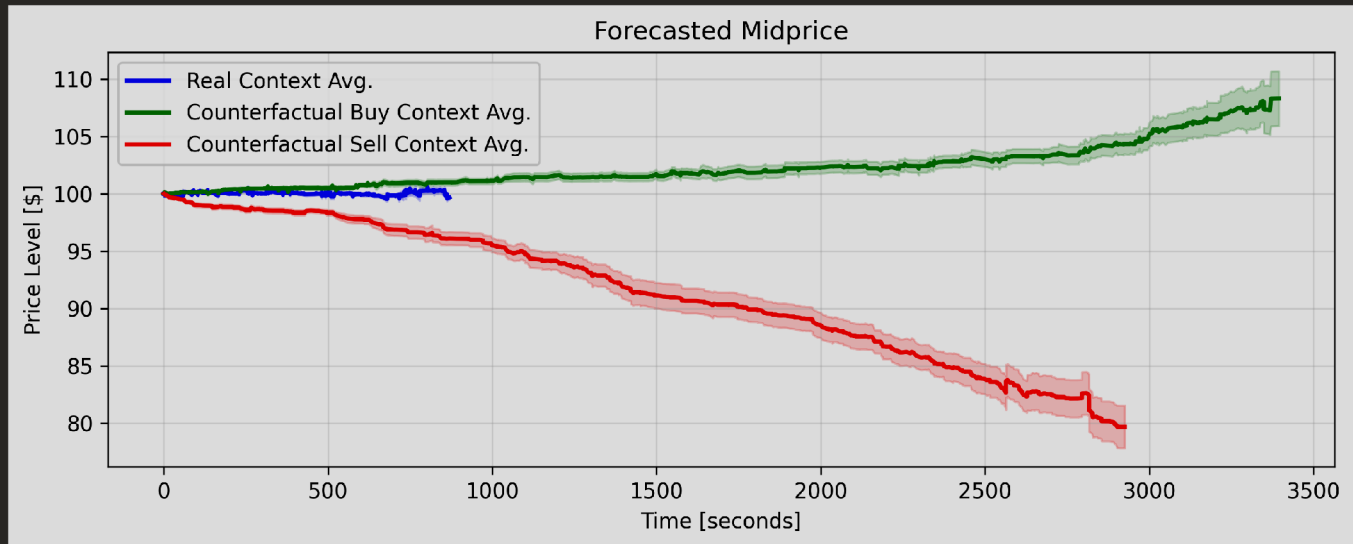


Figure 17: **Counterfactual stress testing.** The model’s generated price path responds realistically to injected anomalous order flow (10x normal frequency), demonstrating its utility for impact analysis.

D.6 Multi-Agent Modeling & RL Fine-Tuning

Our system provides a high-fidelity, interactive environment for training and evaluating sophisticated, learning-based agents. The pre-trained TradeFM can serve as a realistic “background” market, generating plausible and reactive counterparty order flow. This creates a dynamic training ground for:

- **Reinforcement Learning (RL) for Optimal Execution:** RL agents can be trained to learn optimal strategies for executing large orders by interacting with the simulated market, minimizing costs such as price impact and the bid-ask spread.
- **Multi-Agent Systems (MAS):** The simulator can be populated with multiple, heterogeneous learning-based agents to study the emergent, collective behaviors and potential instabilities that arise from their interactions. The participant-level conditioning of our model provides a natural and powerful mechanism for initializing and fine-tuning diverse agent policies within such a system.

Instructions for reporting errors

We are continuing to improve HTML versions of papers, and your feedback helps enhance accessibility and mobile support. To report errors in the HTML that will help us improve conversion and rendering, choose any of the methods listed below:

- Click the "Report Issue" () button, located in the page header.

Tip: You can select the relevant text first, to include it in your report.

Our team has already identified [the following issues](#). We appreciate your time reviewing and reporting rendering errors we may not have found yet. Your efforts will help us improve the HTML versions for all readers, because disability should not be a barrier to accessing research. Thank you for your continued support in championing open access for all.

Have a free development cycle? Help support accessibility at arXiv! Our collaborators at LaTeXML maintain a [list of packages that need conversion](#), and welcome [developer contributions](#).